

EventFlow

As-Built Documentation Pack

26 August 2026

EventFlow — As-Built Documentation Pack

Prepared: 26 August 2026 **Scope:** Full reverse-engineered documentation of the EventFlow application as it exists in the repository at the time of writing (monorepo containing apps/api and apps/web). **Method:** Static analysis of the source code, configuration files, database schema, and automated tests. No runtime instrumentation, penetration testing, or destructive testing was performed.

How to read this pack

Every factual claim in these 24 documents is labelled with one of three evidence tags:

- **CONFIRMED** — directly observed in the source code, schema, configuration, or test files, with a file path cited as evidence.
- **INFERRED** — a reasonable conclusion drawn from code structure or naming, but not explicitly stated or directly tested/verified in the code (e.g. inferring intended UX behavior from a component's props).
- **UNKNOWN** — could not be determined from the available source. Written explicitly as "Not determined from the available source code/project artefacts" rather than guessed.

This pack documents the application **as it is currently built**. It is not a design proposal, a roadmap, or a recommendation of what the application should do. Where something looks unusual or incomplete, it is reported as observed, with any risk noted in 20-known-issues-risks-technical-debt.md rather than silently corrected.

Executive Summary

EventFlow is a two-application monorepo: a TypeScript/Express/Prisma/PostgreSQL API (apps/api) and a React/TypeScript/Vite/Tailwind single-page frontend (apps/web), deployed respectively to Railway and Vercel (CONFIRMED — apps/api/railway.json, apps/web/vercel.json). It is a SaaS product for two related but distinct use cases built on one shared data model:

1. **Private event RSVP and planning** — a subscriber ("planner") creates an event, invites guests via a shareable link/QR code or personalized invitation tokens, tracks RSVPs, assigns guests to tables on a visual drag-and-drop seating planner, manages vendors, sells optional merchandise, and checks guests in at the door.
2. **Public ticketed events** — the same planner can flip an event to isPublic, define one or more paid ticket types, and publish a public listing page (/tickets/:slug) where anyone can buy tickets without an account; tickets are issued as individually scannable QR codes for door check-in.

Both flows share one payments layer supporting three processors — Stripe Connect (USD/GBP), Paystack Subaccounts (NGN), and PayPal (cross-currency) — selected per event, per currency (CONFIRMED — apps/api/src/modules/payouts/payouts.service.ts,

apps/api/prisma/schema.prisma EventPayoutAccount model). A platform fee (default 5%, independently configurable for merchandise vs. tickets) is taken at checkout via each processor's own fee mechanism (CONFIRMED — apps/api/src/config/env.ts).

The application also has an internal ADMIN role for platform support staff, who can browse all subscribers/events, drill into a subscriber's event using the exact same owner-scoped endpoints a planner uses (with every mutation logged to an audit trail), and view cross-platform analytics (CONFIRMED — apps/api/src/modules/admin/*, apps/api/src/middleware/auditAdminEventActions.ts). A public marketing site (landing page, FAQ, admin-editable "Services" cards, and an admin-authored articles/blog system) rounds out the product.

Authentication is JWT-based, delivered via an httpOnly cookie with a bearer-token fallback (CONFIRMED — apps/api/src/middleware/auth.ts, apps/api/src/modules/auth/auth.controller.ts). There are 22 Express route files and 122 distinct API endpoints (see 22-application-inventory.md for the exact count and 09-api-documentation.md for the full list). The Prisma schema defines 27 models. The frontend has 37 page-level components across 9 route groups. Automated test coverage consists of 3 unit test files and 16 integration test files (Vitest + Supertest), run in CI against a real PostgreSQL service container (CONFIRMED — .github/workflows/ci.yml).

The single largest confirmed gap in the current build is the absence of automated tests for the public ticket checkout, ticket issuance, and door-scan check-in flow — this is called out in both 12-testing-documentation.md and 20-known-issues-risks-technical-debt.md.

Document index

#	Document	Contents
01	Application Overview	What EventFlow is, who it's for, high-level capability map, Mermaid context diagram
02	Functional Requirements	FR-XXX requirement statements derived from implemented behaviour
03	User Stories	User stories per persona/module
04	BDD Acceptance Criteria	Gherkin Given/When/Then scenarios for core flows
05	Functional Specification	Module-by-module behavioural detail
06	UI/UX Specification	Screen inventory, navigation, Mermaid user flows
07	System Architecture	5 Mermaid diagrams: high-level, component,

		request/data flow, auth flow, business workflows
08	Database Documentation	ERD + per-table field tables
09	API Documentation	Every endpoint, grouped by module
10	Roles and Permissions	PLANNER vs ADMIN matrix
11	Business Rules	BR-XXX rule statements
12	Testing Documentation	Test strategy, coverage matrix, existing vs. missing scenarios
13	Test Automation Documentation	Frameworks, CI wiring, how to run tests
14	Security Documentation	Implemented controls, potential gaps, unable-to-determine items
15	Deployment & Infrastructure	Railway/Vercel/GitHub Actions/Docker
16	Configuration & Environment	Every env var, secrets redacted
17	Dependencies & Technology Stack	Full dependency inventory
18	User Guide	Planner-facing how-to
19	Administrator Guide	Admin-facing how-to
20	Known Issues, Risks & Technical Debt	Severity-rated, confirmed vs. potential
21	Traceability Matrix	Requirement → code → test cross-reference
22	Application Inventory	Counts: routes, models, pages, components, tests
23	Glossary	Terms and domain vocabulary
24	Documentation Gap Analysis	What this pack could not verify and why

Self-review statement

Every CONFIRMED claim in this pack is backed by a specific file path cited inline, drawn from a direct reading of that file during preparation. Every INFERRED claim is labelled as such and explains the basis for the inference. Where a requested topic could not be established from the repository (e.g. production traffic volume, real incident history, actual

uptime), it is marked UNKNOWN with the standard phrase “Not determined from the available source code/project artefacts” rather than estimated.

01 — Application Overview

Evidence framework: CONFIRMED (cited file path) / INFERRED (reasoned, not directly stated) / UNKNOWN (“Not determined from the available source code/project artefacts”).

1.1 What EventFlow is

EventFlow is a monorepo SaaS application for event planning, guest RSVP management, visual seating planning, vendor tracking, merchandise sales, and public paid ticketing. CONFIRMED — root package.json names the npm workspaces apps/api and apps/web; apps/api/prisma/schema.prisma header comment reads “Event RSVP & Visual Seating Planner - Database Schema”.

The repository contains two applications:

- **apps/api** — a TypeScript Express REST API using Prisma as its ORM against PostgreSQL. CONFIRMED — apps/api/package.json (express, @prisma/client), apps/api/prisma/schema.prisma (provider = "postgresql").
- **apps/web** — a TypeScript React 18 single-page application built with Vite and styled with Tailwind CSS. CONFIRMED — apps/web/package.json (react@^18.3.1, vite@^5.4.8, tailwindcss@^3.4.13).

1.2 Who it is for

Three user personas are directly supported by distinct authentication/authorization paths in the code:

1. **Planner (subscriber)** — the primary paying user. Creates an account (UserRole.PLANNER, the schema default), creates and manages events, guests, seating, vendors, merchandise, and tickets for events they own. CONFIRMED — apps/api/prisma/schema.prisma (enum UserRole { PLANNER ADMIN }, role UserRole @default(PLANNER)).
2. **Admin (internal support staff)** — an internal EventFlow role with UserRole.ADMIN. Can browse all subscribers and events, and can act on any subscriber’s event through the same endpoints a planner uses, via an ownership bypass; every such mutation is written to an audit log. CONFIRMED — apps/api/src/modules/events/events.service.ts (getOwnedEvent), apps/api/src/middleware/auditAdminEventActions.ts.
3. **Guest / ticket buyer (unauthenticated public user)** — never logs in. Reaches the app through one of several unguessable bearer tokens (an event’s rsvpToken, a personal EventInvitation.token, or a public event’s publicSlug) to RSVP, buy merchandise, or buy tickets. CONFIRMED — apps/api/src/modules/rsvp/rsvp.routes.ts, apps/api/src/modules/tickets/checkout.routes.ts.

1.3 Core capability map

Capability	Confirmed by
Account registration/login (email + password, JWT)	apps/api/src/modules/auth/*
Event CRUD, cover image upload	apps/api/src/modules/events/*
Guest list management, CSV import/export, PDF exports	apps/api/src/modules/guests/*
Personalised + shared-link RSVP, QR invites, email sending (Resend)	apps/api/src/modules/rsvp/*, apps/api/src/modules/guests/invite.service.ts
Uploadable, host-designed invitation card (PDF/PNG/JPEG) attached to invite emails	apps/api/src/modules/events/invitationCard.service.ts
Visual drag-and-drop seating planner (tables, seats, venue layout objects)	apps/api/src/modules/seating/*, apps/web/src/pages/events/seating/*
Door check-in (manual + QR scan) for private-event guests	apps/api/src/modules/guests/guests.controller.ts (checkInScan), apps/web/src/pages/events/CheckInTab.tsx
Vendor tracking with multi-currency cost totals	apps/api/src/modules/vendors/*
Merchandise shop (products, guest checkout, stock decrement)	apps/api/src/modules/products/*
Multi-processor payments (Stripe Connect, Paystack, PayPal) with per-event, per-currency payout accounts	apps/api/src/modules/payouts/*, apps/api/src/lib/stripeClient.ts, apps/api/src/lib/paystackClient.ts, apps/api/src/lib/paypalClient.ts
Public ticketed events: ticket types, public listing page, guest checkout, QR ticket issuance, door scan	apps/api/src/modules/tickets/*
In-app notifications (RSVP changes, vendor status, paid orders)	apps/api/src/modules/notifications/*
“Needs Attention” cross-event insights	apps/api/src/modules/insights/*
Per-planner and platform-wide analytics	apps/api/src/modules/analytics/*, apps/api/src/modules/admin/admin.service.ts (getPlatformAnalytics)
Admin support tooling (user/event browser, audit log, payment event log)	apps/api/src/modules/admin/*
Public marketing site: landing page, FAQ, admin-managed “Services” cards, admin-authored articles/blog	apps/web/src/pages/marketing/*, apps/api/src/modules/landing/*, apps/api/src/modules/articles/*
PWA installability (manifest + service	apps/web/public/manifest.webmanifest,

worker + install prompt)

apps/web/public/sw.js,
apps/web/src/components/InstallPromp
t.tsx

1.4 High-level context diagram

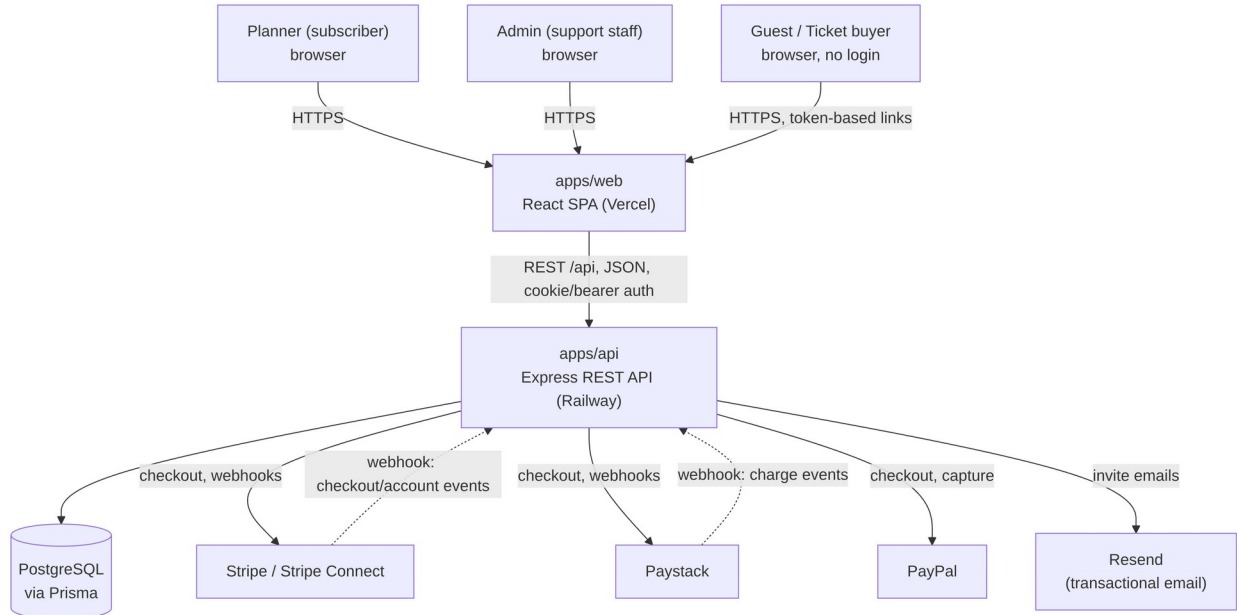


Diagram 1 — 01-application-overview

CONFIRMED (component/relationship existence) — see `apps/api/src/app.ts` for route mounting, `apps/api/src/modules/products/webhook.routes.ts` for inbound webhooks, `apps/api/src/modules/guests/invite.service.ts` for Resend usage. INFERRED — the diagram's visual layout and grouping; the underlying facts are CONFIRMED individually as cited.

1.5 What this document does not cover

Business metrics such as active subscriber counts, revenue, or usage volume are UNKNOWN — Not determined from the available source code/project artefacts. This pack documents the application's built capabilities, not its commercial performance.

02 — Functional Requirements

Requirements below describe behaviour actually implemented in the codebase (CONFIRMED unless marked otherwise), expressed as requirement statements for traceability purposes (see 21-traceability-matrix.md). Each cites its primary evidence file(s).

Authentication & Accounts

- **FR-001:** The system shall allow a new user to register with name, email, and password; email must be a valid address and password must be ≥ 8 characters, containing at least one letter and one number. Evidence: apps/api/src/modules/auth/auth.schema.ts (registerSchema).
- **FR-002:** The system shall reject registration if the email is already in use. Evidence: apps/api/src/modules/auth/auth.service.ts (registerUser $\rightarrow$ ConflictError).
- **FR-003:** The system shall store passwords only as bcrypt hashes (12 salt rounds). Evidence: apps/api/src/utls/password.ts.
- **FR-004:** The system shall allow a registered user to log in with email/password and receive a signed JWT delivered as an httpOnly cookie, with the same token also returned in the response body for bearer-token use. Evidence: apps/api/src/modules/auth/auth.controller.ts.
- **FR-005:** The system shall allow a logged-in user to log out, clearing the session cookie. Evidence: apps/api/src/modules/auth/auth.controller.ts (logout).
- **FR-006:** The system shall expose the current authenticated user's profile via GET /api/auth/me. Evidence: apps/api/src/modules/auth/auth.routes.ts.
- **FR-007:** The system shall rate-limit registration and login to 30 requests per 15 minutes per client. Evidence: apps/api/src/modules/auth/auth.routes.ts.

Event Management

- **FR-010:** The system shall allow a planner to create an event with name, type, date, and optional description, times, venue, capacity, image, RSVP deadline, and per-field RSVP toggles. Evidence: apps/api/src/modules/events/events.schema.ts, events.service.ts (createEvent).
- **FR-011:** The system shall generate a unique, unguessable rsvpToken for every event at creation. Evidence: apps/api/prisma/schema.prisma (Event.rsvpToken @unique @default(cuid())).
- **FR-012:** The system shall allow a planner to update or delete only events they own; an ADMIN may update (but never delete) another subscriber's event. Evidence: apps/api/src/modules/events/events.service.ts (getOwnedEvent vs. getOwnedEventStrict, used by deleteEvent).
- **FR-013:** The system shall allow a planner to upload a PNG/JPEG/WEBP cover image up to 5MB per event, stored as bytes in the database. Evidence: apps/api/src/modules/events/events.service.ts (uploadEventCoverImage).

- **FR-014:** The system shall compute and return a per-event dashboard of guest/table/RSVP/dietary/check-in statistics. Evidence: apps/api/src/modules/events/events.service.ts (getEventDashboard).
- **FR-015:** The system shall list a planner's events with an attached lightweight guest/table summary per event. Evidence: apps/api/src/modules/events/events.service.ts (listEvents).
- **FR-016:** The system shall allow a planner to mark an event isPublic, generating an immutable, unique publicSlug the first time it is made public. Evidence: apps/api/src/modules/events/events.service.ts (updateEvent, ensureUniqueEventSlug).

Guest Management

- **FR-020:** The system shall allow a planner to add, edit, and delete guests on an event they own, including named accompanying party members (" + 1"s). Evidence: apps/api/src/modules/guests/guests.service.ts.
- **FR-021:** The system shall allow searching/filtering the guest list by name/email/group, RSVP status, seat-assignment state, check-in state, VIP flag, and presence of dietary requirements. Evidence: apps/api/src/modules/guests/guests.schema.ts (listGuestsQuerySchema), guests.service.ts (listGuests).
- **FR-022:** The system shall support bulk guest import via CSV upload. Evidence: apps/api/src/modules/guests/guests.routes.ts (POST /import), guests.service.ts (bulkCreateGuests).
- **FR-023:** The system shall support exporting the guest list as CSV and as PDF, and exporting printable QR wristbands/badges as PDF. Evidence: apps/api/src/modules/guests/guests.routes.ts (/export, /export/pdf, /wristbands/pdf).
- **FR-024:** The system shall allow a planner to manually check a guest in/out, and to check a guest in by scanning their invitation QR code. Evidence: apps/api/src/modules/guests/guests.service.ts (checkInGuest, checkOutGuest), invite.service.ts (checkInGuestByToken).
- **FR-025:** The system shall free a guest's (and their party's) seat assignment automatically if their RSVP status moves away from CONFIRMED. Evidence: apps/api/src/utils/rsvpMath.ts (shouldReleaseSeatOnStatusChange), guests.service.ts (updateGuest).

RSVP (Guest-Facing, Public)

- **FR-030:** The system shall serve a public RSVP page for any valid event rsvpToken, without login. Evidence: apps/api/src/modules/rsvp/rsvp.routes.ts, rsvp.service.ts (getPublicEventByToken).
- **FR-031:** The system shall accept RSVP submissions (attending status, party names, meal/dietary/accessibility/notes as enabled per event) via the shared token and via a personalised invitation token, matching an existing invited guest by email then by

name where possible. Evidence: apps/api/src/modules/rsvp/rsvp.service.ts (submitRsvp, submitRsvpViaInvitation).

- **FR-032:** The system shall reject RSVP submissions once the event's RSVP deadline has passed or rsvpOpen is false. Evidence: apps/api/src/modules/rsvp/rsvp.service.ts (checkRsvpIsOpen).
- **FR-033:** The system shall rate-limit RSVP submission (default 20/15 min, configurable) and RSVP reads (60/min). Evidence: apps/api/src/modules/rsvp/rsvp.routes.ts.
- **FR-034:** The system shall notify the event owner in-app when a guest confirms or declines. Evidence: apps/api/src/modules/notifications/notifications.service.ts (notifyRsvpChange).

Invitations

- **FR-040:** The system shall generate a per-guest personalised invitation link and QR code on demand. Evidence: apps/api/src/modules/guests/invite.service.ts (getInviteLink).
- **FR-041:** The system shall send invitation emails (with embedded QR and optional uploaded invitation card attachment) via Resend, when configured. Evidence: apps/api/src/modules/guests/invite.service.ts (sendInviteEmail, getResendClient).
- **FR-042:** The system shall support bulk-sending invite emails to all or selected guests with an email address. Evidence: apps/api/src/modules/guests/invite.service.ts (bulkSendInviteEmails).
- **FR-043:** The system shall allow a planner to upload, replace, and delete a single designed invitation card (PDF/PNG/JPEG, ≤8MB) per event. Evidence: apps/api/src/modules/events/invitationCard.service.ts.

Seating Planner

- **FR-050:** The system shall provide a venue layout (canvas size, grid, background) with placeable decor objects (stage, bar, dance floor, etc.) per event, auto-created on first access. Evidence: apps/api/src/modules/seating/seating.service.ts (getOrCreateLayout).
- **FR-051:** The system shall support creating tables of various shapes with a defined seat capacity, auto-generating one Seat row per capacity unit. Evidence: apps/api/src/modules/seating/seating.service.ts (createTable).
- **FR-052:** The system shall support assigning a confirmed guest (and, automatically, their named party members to nearby seats) to a table, enforcing capacity unless explicitly overridden. Evidence: apps/api/src/modules/seating/seating.service.ts (assignGuest), apps/api/src/utils/capacity.ts (canAssignGuest).
- **FR-053:** The system shall, when a table's capacity is reduced below its currently seated count, unseat the highest-numbered seats' occupants (whole party, not partial) and report who was unseated. Evidence: apps/api/src/modules/seating/seating.service.ts (updateTable).

- **FR-054:** The system shall support unassigning a whole guest party, or a single named party member, from their seat independently. Evidence: apps/api/src/modules/seating/seating.service.ts (unassignGuest, unassignPartyMember).
- **FR-055:** The system shall export the seating map as a PDF. Evidence: apps/api/src/modules/seating/seating.routes.ts (GET /map/export/pdf).

Vendors

- **FR-060:** The system shall allow a planner to track vendors per event with category, status, contact info, cost, currency, and deposit-paid flag. Evidence: apps/api/src/modules/vendors/vendors.service.ts.
- **FR-061:** The system shall report vendor cost totals grouped by currency rather than a single blended sum. Evidence: apps/api/src/modules/vendors/vendors.service.ts (groupCostsByCurrency, getVendorSummary).
- **FR-062:** The system shall notify the event owner when a vendor's status changes. Evidence: apps/api/src/modules/notifications/notifications.service.ts (notifyVendorStatusChanged).

Merchandise (Event Shop)

- **FR-070:** The system shall allow a planner to create products with name, description, price, currency, stock quantity (or unlimited), active flag, and an optional image. Evidence: apps/api/src/modules/products/products.service.ts.
- **FR-071:** The system shall expose an "shop enabled" toggle per event (merchandiseEnabled) gating whether guests see the shop at all. Evidence: apps/api/prisma/schema.prisma (Event.merchandiseEnabled), products.service.ts (listPublicProducts).
- **FR-072:** The system shall let a guest add active, in-stock products to a cart and check out via any connected, currency-matching payment provider. Evidence: apps/api/src/modules/products/orders.service.ts (createCheckoutSession).
- **FR-073:** The system shall reject a cart mixing items priced in different currencies. Evidence: apps/api/src/modules/products/orders.service.ts (createCheckoutSession).
- **FR-074:** The system shall decrement stock only when an order is confirmed PAID by the processor, not at checkout initiation. Evidence: apps/api/src/modules/products/orders.service.ts (finalizeOrderPaid).

Payments / Payouts

- **FR-080:** The system shall let a planner connect a Stripe Express account (USD/GBP), a Paystack Subaccount (NGN), and/or a PayPal payee email (any of the three currencies) as payout destinations, per event, per currency. Evidence: apps/api/src/modules/payouts/payouts.service.ts.
- **FR-081:** The system shall route guest checkout to a specific connected+ready payout account, defaulting to a fixed preference order (Stripe Connect → Paystack → PayPal) when the guest doesn't choose. Evidence:

apps/api/src/modules/products/orders.service.ts
(DEFAULT_PROVIDER_PREFERENCE).

- **FR-082:** The system shall capture a platform fee on every paid order via the processor's own fee mechanism, at a percentage read from configuration at checkout time and frozen onto the order thereafter. Evidence: apps/api/src/config/env.ts, apps/api/prisma/schema.prisma (Order.platformFeeCents).
- **FR-083:** The system shall verify Stripe and Paystack webhook signatures before trusting any payment-confirmation payload. Evidence: apps/api/src/modules/products/orders.service.ts (handleStripeWebhook, handlePaystackWebhook).
- **FR-084:** The system shall log every payment provider event (success, failure, expiry) to a queryable PaymentEvent record, independent of order status. Evidence: apps/api/src/modules/products/orders.service.ts (logPaymentEvent).

Ticketing (Public Events)

- **FR-090:** The system shall let a planner define one or more ticket types (price, currency, capacity, sales window, min/max per order) for a public event. Evidence: apps/api/src/modules/tickets/ticketTypes.service.ts.
- **FR-091:** The system shall serve a public, unauthenticated ticket-purchase page keyed by an event's publicSlug. Evidence: apps/api/src/modules/tickets/checkout.service.ts (getPublicTicketEvent).
- **FR-092:** The system shall atomically reserve ticket-type capacity at the moment a checkout session is created (not at payment confirmation), to prevent overselling under concurrent checkouts. Evidence: apps/api/src/modules/tickets/ticketTypes.service.ts (reserveTickets, raw SQL UPDATE ... WHERE).
- **FR-093:** The system shall release reserved capacity if a ticket checkout expires, is declined, or is cancelled before payment. Evidence: apps/api/src/modules/products/orders.service.ts (releasePendingTicketOrder).
- **FR-094:** The system shall issue one individually-scannable Ticket row (unique code) per unit purchased once an order is confirmed PAID. Evidence: apps/api/src/modules/products/orders.service.ts (finalizeOrderPaid).
- **FR-095:** The system shall support door check-in of a ticket by scanning its code, idempotently (re-scanning an already-checked-in ticket does not error but is flagged). Evidence: apps/api/src/modules/tickets/ticketTypes.service.ts (checkInTicketByCode).

Notifications & Insights

- **FR-100:** The system shall generate in-app notifications for RSVP changes, vendor status changes, and paid orders (merchandise or ticket). Evidence: apps/api/src/modules/notifications/notifications.service.ts.
- **FR-101:** The system shall allow marking one or all notifications as read. Evidence: apps/api/src/modules/notifications/notifications.routes.ts.

- **FR-102:** The system shall compute “Needs Attention” insights on demand (approaching RSVP deadlines, unassigned VIPs/confirmed guests, missing meal selections, pending responses) without persisting them. Evidence: apps/api/src/modules/insights/insights.service.ts.

Analytics

- **FR-110:** The system shall provide a planner-level cross-event analytics overview (RSVP funnel, check-in rate, vendor spend by currency, per-event breakdown). Evidence: apps/api/src/modules/analytics/analytics.service.ts.
- **FR-111:** The system shall provide a platform-wide analytics rollup for admins (subscriber/event/guest counts, RSVP confirmation rate, revenue by currency and provider, 30-day signup/event trend). Evidence: apps/api/src/modules/admin/admin.service.ts (getPlatformAnalytics).

Admin / Support

- **FR-120:** The system shall restrict /api/admin/* routes to authenticated users with role = ADMIN. Evidence: apps/api/src/middleware/auth.ts (requireAdmin).
- **FR-121:** The system shall let an admin list all subscribers and all events across the platform. Evidence: apps/api/src/modules/admin/admin.service.ts (listAllUsers, listAllEvents).
- **FR-122:** The system shall let an admin act on a subscriber’s event through the same endpoints the subscriber uses, without a separate admin-only event API. Evidence: apps/api/src/modules/events/events.service.ts (getOwnedEvent admin bypass).
- **FR-123:** The system shall write an audit log entry for every non-GET, successful request an admin makes against an event they do not own. Evidence: apps/api/src/middleware/auditAdminEventActions.ts.
- **FR-124:** The system shall prevent an admin from deleting a subscriber’s event or from creating/changing a subscriber’s payout account details, even in support mode. Evidence: apps/api/src/modules/events/events.service.ts (getOwnedEventStrict, used by deleteEvent), apps/api/src/modules/payouts/payouts.service.ts (connectStripe/connectPaystack/connectPaypal all call getOwnedEventStrict).

Public Content (Marketing)

- **FR-130:** The system shall serve a public marketing landing page including an FAQ section and admin-managed “Services” cards. Evidence: apps/web/src/pages/marketing/LandingPage.tsx, apps/api/src/modules/landing/*.
- **FR-131:** The system shall let an admin author, edit, publish/unpublish, and delete blog articles, each with an auto-generated immutable slug. Evidence: apps/api/src/modules/articles/articles.service.ts.
- **FR-132:** The system shall serve published articles publicly at /articles and /articles/:slug. Evidence: apps/api/src/modules/articles/articles.public.routes.ts.

Platform / Non-functional

- **FR-140:** The system shall expose a /health endpoint that checks database connectivity. Evidence: apps/api/src/app.ts.
- **FR-141:** The system shall be installable as a Progressive Web App (manifest + service worker + install prompt). Evidence: apps/web/public/manifest.webmanifest, apps/web/public/sw.js, apps/web/src/components/InstallPrompt.tsx.
- **FR-142:** The system shall apply security headers (Helmet) and restrict CORS to a configured origin list. Evidence: apps/api/src/app.ts.

03 — User Stories

Format: As a <persona>, I want <capability>, so that <benefit>. Each story is INFERRED in wording (user stories are a documentation convention, not literally present in code) but is grounded in CONFIRMED implemented behaviour, cited per story. Story IDs map to the functional requirements in 02-functional-requirements.md and the traceability matrix in 21-traceability-matrix.md.

Planner — Account & Events

- **US-001:** As a planner, I want to register an account with my email and a password, so that I can start creating events. → FR-001.
apps/api/src/modules/auth/auth.service.ts.
- **US-002:** As a planner, I want to log in and stay signed in across visits, so that I don't have to re-authenticate constantly. → FR-004.
apps/api/src/modules/auth/auth.controller.ts (7-day cookie).
- **US-003:** As a planner, I want to create an event with a name, type, date, and venue, so that I have a place to manage everything about it. → FR-010.
- **US-004:** As a planner, I want to see all my events in one dashboard with at-a-glance RSVP/table progress, so that I can quickly tell which events need attention. → FR-015.
apps/web/src/pages/events/EventsListPage.tsx.
- **US-005:** As a planner, I want to upload a cover image for my event, so that invites and the event page look polished. → FR-013.

Planner — Guests & RSVP

- **US-010:** As a planner, I want to add guests one at a time or import a whole list from a CSV, so that I don't have to type in every guest by hand. → FR-020, FR-022.
- **US-011:** As a planner, I want to share one RSVP link (or QR code) for the whole event, so that guests can respond without me sending individual invites. → FR-030.
apps/api/src/modules/rsvp/rsvp.service.ts (getPublicEventByToken).
- **US-012:** As a planner, I want to send a personalised invite (with its own QR code) to a specific guest, so that their RSVP is pre-filled with their name and I know exactly who responded. → FR-040, FR-041.
- **US-013:** As a planner, I want an uploaded, professionally designed invitation card attached to invite emails, so that the invite matches my event's branding instead of a plain text email. → FR-043.
- **US-014:** As a planner, I want to see who hasn't responded yet, so that I can follow up before the deadline. → apps/api/src/modules/rsvp/rsvp.service.ts (getRsvpDashboard, nonResponders).
- **US-015:** As a planner, I want to export my guest list and printable QR wristbands as PDFs, so that I have physical materials for the event day. → FR-023.

Planner — Seating

- **US-020:** As a planner, I want to drag guests onto a visual map of tables, so that I can plan seating the way I'd sketch it on paper. → FR-051, FR-052.
apps/web/src/pages/events/seating/SeatingCanvas.tsx.
- **US-021:** As a planner, I want a guest's "+1"s to automatically get seats near them, so that families/couples aren't scattered across the room. → FR-052 (party seating in assignGuest).
- **US-022:** As a planner, I want to be warned (not silently blocked) if I try to overbook a table, so that I retain control for edge cases like last-minute extra chairs. → apps/api/src/utils/capacity.ts (canAssignGuest, overrideCapacity).
- **US-023:** As a planner, I want to shrink a table's capacity and be told exactly who got unseated, so that I can immediately reseal them elsewhere. → FR-053.

Planner — Vendors & Merchandise

- **US-030:** As a planner, I want to track every vendor I'm working with, their status, and what I owe them, so that nothing falls through the cracks. → FR-060.
- **US-031:** As a planner paying vendors in more than one currency, I want my cost totals broken out per currency, so that a blended number doesn't mislead me about what I actually owe. → FR-061.
- **US-032:** As a planner, I want to sell branded merchandise directly from my event page, so that I have an extra revenue stream without a separate storefront. → FR-070, FR-072.

Planner — Payments

- **US-040:** As a planner, I want to connect my own Stripe/Paystack/PayPal account, so that guest payments go straight to me (minus a platform fee), not through a middleman I don't control. → FR-080.
- **US-041:** As a planner selling in NGN, I want a payout option that actually supports Nigerian bank accounts, so that I'm not stuck with a processor that doesn't serve my country. → FR-080 (Paystack Subaccount for NGN specifically).

Planner — Ticketing

- **US-050:** As a planner running a public event (e.g. a club night), I want to define multiple ticket tiers with different prices and quantities, so that I can offer early-bird/VIP/general pricing. → FR-090.
- **US-051:** As a planner, I want a shareable public event page anyone can buy tickets from without creating an account, so that I don't add friction to impulse ticket purchases. → FR-091.
- **US-052:** As a planner, I want ticket capacity to never be oversold even if many people check out at once, so that I don't have to turn away a paying guest at the door. → FR-092.
- **US-053:** As door staff, I want to scan a ticket's QR code and get an instant valid/already-used/invalid result, so that entry is fast and I can't be fooled by a shared screenshot. → FR-095.

Planner — Check-in

- **US-060:** As door staff, I want to scan a guest's invitation QR code to check them in, so that check-in doesn't require typing names into a list. → FR-024 (checkInGuestByToken).
- **US-061:** As door staff, I want to check a guest in manually by name if the QR code fails to scan, so that a technical hiccup doesn't stop the line. → FR-024 (checkInGuest).

Planner — Analytics & Notifications

- **US-070:** As a planner running multiple events, I want a single view of RSVP/check-in performance across all of them, so that I don't have to open each event individually. → FR-110.
- **US-071:** As a planner, I want to be notified in-app the moment a guest RSVPs, a vendor's status changes, or an order is paid, so that I stay on top of activity without refreshing every tab. → FR-100.
- **US-072:** As a planner, I want a "Needs Attention" list surfacing unassigned VIPs and approaching deadlines, so that I know what to act on today, not just raw data. → FR-102.

Admin (Support Staff)

- **US-080:** As an admin, I want to see every subscriber and event on the platform, so that I can find and assist a specific customer quickly. → FR-121.
- **US-081:** As an admin, I want to open a subscriber's event and see/do exactly what they see, so that I can reproduce and fix their issue directly instead of guessing from a description. → FR-122.
- **US-082:** As an admin, I want every action I take on someone else's event logged automatically, so that there's an accountable record of support activity. → FR-123.
- **US-083:** As an admin, I want platform-wide analytics (signups, revenue by currency/provider), so that I can report on business health without querying the database by hand. → FR-111.
- **US-084:** As an admin, I want to author and publish blog articles and manage the landing page's Services cards, so that marketing content can be updated without a code deploy. → FR-131, FR-130.

Guest / Ticket Buyer (Public, Unauthenticated)

- **US-090:** As a guest, I want to open a link and RSVP without creating an account, so that responding is effortless. → FR-030, FR-031.
- **US-091:** As a guest with dietary needs, I want to specify my meal preference and any allergies when I RSVP, so that the host can plan catering correctly. → FR-031 (allowMealSelection/allowDietary toggles).
- **US-092:** As a guest, I want to buy event merchandise and pay in a currency the event actually accepts, so that checkout doesn't fail with a confusing error. → FR-072, FR-073.

- **US-093:** As a member of the public, I want to discover and buy tickets to a public event without needing an invitation, so that I can attend events I hear about through a shared link. → FR-091.

04 — BDD Acceptance Criteria

Gherkin scenarios describing implemented behaviour, derived from reading the service-layer logic (INFERRED as test-style prose; the underlying business rules are CONFIRMED against the cited files). These are **not** verbatim from an existing .feature file — the codebase has no Cucumber/Gherkin test runner (see 13-test-automation-documentation.md). Where an automated test already exercises the same behaviour, it is noted.

Feature: Guest RSVP submission

Feature: Public guest RSVP

Evidence: apps/api/src/modules/rsvp/rsvp.service.ts

Scenario: Guest confirms attendance before the deadline

Given an event with rsvpOpen = true and no rsvpDeadline in the past

And a guest opens the event's public RSVP link

When the guest submits "CONFIRMED" with their name and meal preference

Then a Guest record is created or updated with rsvpStatus = CONFIRMED

And the event owner receives an RSVP_CONFIRMED notification

Covered by: apps/api/tests/integration/rsvp.test.ts

Scenario: Guest attempts to RSVP after the deadline

Given an event whose rsvpDeadline has passed

When a guest submits an RSVP

Then the request is rejected with a 400 Bad Request

And no Guest record is modified

Evidence: rsvp.service.ts checkRsvpIsOpen()

Scenario: Guest un-confirms after previously being seated

Given a guest is CONFIRMED and assigned to a seat

When the guest resubmits their RSVP as "DECLINED"

Then their rsvpStatus becomes DECLINED

And their SeatingAssignment (and their party's) is deleted

Evidence: rsvp.service.ts submitRsvp(), utils/rsvpMath.ts
shouldReleaseSeatOnStatusChange()

Feature: Seating assignment with capacity rules

Feature: Seat a guest at a table

Evidence: apps/api/src/modules/seating/seating.service.ts,
apps/api/src/utils/capacity.ts

Scenario: Assign a confirmed guest to a table with room

Given a table with capacity 8 and 5 seats currently occupied

And a confirmed guest with no accompanying party members

When the planner assigns the guest to that table

Then the assignment succeeds

And the guest occupies exactly 1 of the table's seats

Covered by: apps/api/tests/integration/seating.test.ts,
apps/api/tests/unit/capacity.test.ts

Scenario: Assign a guest whose party would overflow the table
Given a table with 2 seats remaining
And a confirmed guest with 2 named party members (party size 3)
When the planner assigns the guest without overriding capacity
Then the assignment is rejected with a reason describing the party-size shortfall
Evidence: capacity.ts canAssignGuest()

Scenario: Planner overrides capacity intentionally
Given a table already at full capacity
When the planner assigns a guest with overrideCapacity = true
Then the assignment succeeds
And a warning "Table is now over capacity" is returned
Evidence: capacity.ts canAssignGuest()

Scenario: Shrinking a table unseats the highest-numbered occupied seats
Given a table with 8 seats, seats 7 and 8 occupied
When the planner reduces capacity to 6
Then seats 7 and 8 are removed
And the guests who occupied them (and their whole party) are unassigned, not deleted
And the response lists the unseated guests' names
Evidence: seating.service.ts updateTable()

Feature: Merchandise checkout

Feature: Guest purchases merchandise
Evidence: apps/api/src/modules/products/orders.service.ts

Scenario: Successful checkout via Stripe Connect
Given an event with merchandiseEnabled = true
And a connected, onboarding-complete Stripe payout account for USD
And a guest has items in their cart priced in USD
When the guest starts checkout
Then a PENDING Order is created with the computed platform fee
And a Stripe Checkout Session is created with application_fee_amount set
And the guest is redirected to Stripe's hosted checkout
Covered by: apps/api/tests/integration/merchandise.test.ts

Scenario: Cart mixes currencies
Given a cart containing one USD-priced item and one GBP-priced item
When the guest attempts checkout
Then the request is rejected with a message about mixed currencies
Evidence: orders.service.ts createCheckoutSession()

Scenario: No payment provider connected for the cart's currency
Given an event with no EventPayoutAccount connected for NGN
When a guest attempts checkout with an NGN-priced cart

Then the request is rejected explaining NGN payments aren't accepted yet
Evidence: orders.service.ts createCheckoutSession()

Scenario: Stripe confirms payment via webhook
Given a PENDING order tied to a Stripe Checkout Session
When Stripe sends a verified checkout.session.completed webhook event
Then the order status becomes PAID
And the purchased product's stockQuantity is decremented
And the event owner receives an ORDER_PAID notification
Evidence: orders.service.ts handleStripeWebhook(), finalizeOrderPaid()

Feature: Ticket checkout and capacity reservation

Feature: Public ticket purchase

Evidence: apps/api/src/modules/tickets/checkout.service.ts,
apps/api/src/modules/tickets/ticketTypes.service.ts

Scenario: Two buyers race for the last ticket
Given a ticket type with 1 ticket remaining
When two checkout requests for that ticket type are submitted concurrently
Then exactly one reservation succeeds
And the other is rejected with "Not enough tickets remaining"
Evidence: ticketTypes.service.ts reserveTickets() — single conditional UPDATE,
not read-then-write
Automated test coverage: NOT FOUND for this specific concurrency scenario —
see 12-testing-documentation.md

Scenario: Ticket order abandoned before payment
Given a PENDING ticket order that reserved capacity
When the Stripe Checkout Session expires without payment
Then the reserved capacity is released back to the ticket type
And the order is marked CANCELLED
Evidence: orders.service.ts releasePendingTicketOrder(),
handleStripeWebhook() (checkout.session.expired)

Scenario: Ticket order confirmed paid
Given a PENDING ticket order
When the processor confirms payment
Then one Ticket row (unique code) is created per unit purchased
And the buyer's confirmation page can retrieve them via the order id
Evidence: orders.service.ts finalizeOrderPaid(), tickets/checkout.service.ts
getPublicTicketOrder()

Feature: Door check-in by QR scan

Feature: Ticket door check-in

Evidence: apps/api/src/modules/tickets/ticketTypes.service.ts
checkInTicketByCode()

Scenario: First scan of a valid ticket
Given a Ticket with status VALID

When staff scan its code
Then the ticket status becomes CHECKED_IN
And alreadyCheckedIn = false is returned

Scenario: Re-scan of an already checked-in ticket
Given a Ticket with status CHECKED_IN
When staff scan its code again
Then no error occurs
And alreadyCheckedIn = true is returned, flagging a possible duplicate/shared ticket

Scenario: Scan of a cancelled ticket
Given a Ticket with status CANCELLED
When staff scan its code
Then the request is rejected explaining the ticket can't be used for entry

Feature: Admin support access and audit logging

Feature: Admin drill-in to a subscriber's event

Evidence: apps/api/src/modules/events/events.service.ts getOwnedEvent(),
apps/api/src/middleware/auditAdminEventActions.ts

Scenario: Admin reads a subscriber's event
Given a user with role ADMIN who does not own event E
When they issue GET /api/events/:E
Then the request succeeds (not 403/404)
And no audit log entry is written (GET requests are not audited)

Scenario: Admin edits a subscriber's guest
Given a user with role ADMIN who does not own event E
When they issue PUT /api/events/:E/guests/:guestId
And the request succeeds
Then an AdminAuditLog row is written recording the admin's email, event, method, and a human-readable summary

Scenario: Admin attempts to delete a subscriber's event
Given a user with role ADMIN who does not own event E
When they issue DELETE /api/events/:E
Then the request is rejected with 404 Not Found
Evidence: events.service.ts deleteEvent() calls getOwnedEventStrict(), which has no admin bypass

Scenario: Non-admin planner attempts to reach /api/admin
Given a user with role PLANNER
When they issue any request to /api/admin/*
Then the request is rejected with 403 Forbidden
Evidence: middleware/auth.ts requireAdmin()

05 — Functional Specification

Module-by-module behavioural detail. All statements CONFIRMED against the cited file unless marked INFERRED/UNKNOWN.

5.1 Auth module (`apps/api/src/modules/auth`)

Files: `auth.routes.ts`, `auth.controller.ts`, `auth.service.ts`, `auth.schema.ts`.

- Registration (POST `/api/auth/register`): validates via Zod (`registerSchema`), rejects duplicate email with 409 CONFLICT, hashes password with `bcrypt` (12 rounds), signs a JWT `{userId}` with `env.jwtExpiresIn` (default 7d), sets it as an `httpOnly` cookie (`event_rsvp_token` by default) and returns it in the JSON body as well.
- Login (POST `/api/auth/login`): looks up by email, 401 UNAUTHORIZED (“Invalid email or password”) for both a missing user and a wrong password — deliberately not distinguished, to avoid user enumeration. INFERRED intent (not stated in a comment, but the identical error message for both cases is the CONFIRMED code behaviour).
- Logout (POST `/api/auth/logout`): clears the cookie; does not require authentication to call (no `requireAuth` on this route) and does not invalidate the JWT server-side — a previously-issued token/cookie value remains cryptographically valid until its 7-day expiry even after “logout” (see `14-security-documentation.md`).
- GET `/api/auth/me`: requires auth, returns the current user’s `{id, email, name, role, createdAt}`.
- Cookie attributes: `httpOnly: true`, `secure: env.isProduction`, `sameSite: "none"` in production / `"lax"` in development, `maxAge: 7 days`, `path: "/"`.

5.2 Events module (`apps/api/src/modules/events`)

Files: `events.routes.ts`, `events.controller.ts` (not read line-by-line this pass but routes confirm shape), `events.service.ts`, `events.schema.ts`, `invitationCard.service.ts`.

- `getOwnedEvent(userId, eventId)` is the single ownership gate reused by every nested module (guests, seating, vendors, products, orders, payouts, ticket types, invitation card). It returns the event if `event.userId === userId`, OR if the requesting user’s role is ADMIN — otherwise throws `NotFoundError` (never `ForbiddenError`, so an admin-blocked or non-existent event ID look identical to a prober).
- `getOwnedEventStrict` is the same check with no admin bypass, used only for: deleting an event, and connecting/changing a payout account (Stripe/Paystack/PayPal).
- Event creation defaults: `allowPlusOnes`, `allowPlusOneNames`, `allowMealSelection`, `allowDietary`, `allowAccessibilityInfo`, `allowSpecialRequests` all default true unless specified.
- `publicSlug` is generated once (slugified from the event name, deduplicated with a -2, -3, ... suffix on collision) the first time `isPublic` is set true, and is never regenerated afterward even if the event name later changes.

- Cover image upload accepts PNG/JPEG/WEBP up to 5MB, stored as Bytes in Postgres (coverImageData/coverImageMimeType columns), not an external object store.
- getEventDashboard computes 13 distinct counts in parallel (Promise.all) plus two follow-up queries (unassigned confirmed count, party-size aggregate) — 15 total queries per dashboard load. INFERRED performance note: this is a repository-level micro-optimization observation, not a stated design comment; see 20-known-issues-risks-technical-debt.md for whether this is flagged as a concern.

5.3 Guests module (apps/api/src/modules/guests)

Files: guests.routes.ts (nested), guestById.routes.ts (top-level /api/guests/:guestId), guests.service.ts, guests.schema.ts, invite.service.ts.

- Two mount points exist for guest-by-ID operations: nested under /api/events/:eventId/guests/:guestId and top-level at /api/guests/:guestId (the latter requires requireAuth explicitly since it isn't behind the events router's auth). Both ultimately call the same guests.service.ts functions.
- listGuests supports search (name/email/group, case-insensitive contains), status filter, assigned/unassigned filter, checked-in filter, VIP filter, and "has dietary requirements" filter, plus sort by last/first name, RSVP status, or creation date.
- createGuest/updateGuest accept an additionalGuestNames array; if provided, additionalGuestsCount is derived from its length (overriding any explicitly passed count) and a GuestParty row is created per name. updateGuest fully replaces the party list (delete-all, re-create) rather than diffing.
- Check-in writes to two places atomically in one transaction: the Guest.checkedIn/checkedInAt flags (used everywhere else in the app) and a CheckIn audit row (upsert, so it's idempotent per guest) recording who checked them in.
- CSV import (bulkCreateGuests) creates all rows in one \$transaction array — an all-or-nothing batch insert with no per-row validation feedback beyond whatever Zod validation happens before the call (INFERRED from guests.routes.ts: upload.single("file") – controller.importCsv, CSV parsing itself lives in the controller, not reviewed line-by-line this pass — UNKNOWN whether malformed individual rows are skipped or fail the whole batch; flagged in 24-documentation-gap-analysis.md).

5.4 RSVP module (apps/api/src/modules/rsvp)

Files: rsvp.routes.ts, rsvp.service.ts, rsvp.schema.ts.

- Fully public/unauthenticated router, mounted at /api/rsvp. No requireAuth anywhere in this file.
- Two entry paths: /api/rsvp/:token (shared event-level rsvpToken) and /api/rsvp/invite/:invitationToken (personalised). The invite-token path resolves directly to a specific guest with no name/email matching ambiguity; the shared-token path attempts to match an existing invited guest by email (case-insensitive), then by

exact first+last name (case-insensitive), before falling back to creating a new Guest row.

- `checkRsvpIsOpen` blocks submission if `event.rsvpOpen` is false OR `rsvpDeadline` has passed — both reads happen fresh per request (no caching), so a deadline passing mid-session is enforced on the next write.
- Rate limits: submission at `env.rsvpRateLimitMax/env.rsvpRateLimitWindowMs` (defaults 20 requests / 15 minutes), reads at a fixed 60/minute — both keyed by client IP (express-rate-limit default keying; INFERRED, standard library default, not overridden in this file).
- On any RSVP status change away from CONFIRMED, seating assignments are deleted within the same transaction as the guest update.

5.5 Seating module (`apps/api/src/modules/seating`)

Files: `seating.routes.ts`, `seating.service.ts`, `seating.schema.ts`.

- Venue layout: one `VenueLayout` per event, auto-created with defaults (`canvasWidth`: 1600, `canvasHeight`: 1000, `gridSize`: 20, `backgroundColor`: "#f8fafc") on first GET/PUT if it doesn't exist yet. Decor objects (`LayoutObject`) are freeform positioned/sized/rotated/coloured shapes typed from a fixed enum (`STAGE`, `DANCE_FLOOR`, `BAR`, `BUFFET`, `ENTRANCE`, `EXIT`, `TOILETS`, `DJ_BOOTH`, `VIP_AREA`, `CUSTOM`).
- Tables: creating a table with capacity: `N` creates exactly `N` Seat rows numbered 1..`N`. Table shapes: `ROUND` | `SQUARE` | `RECTANGLE` | `OVAL` | `BANQUET` | `HEAD` | `CUSTOM`.
- `assignGuest` is the most complex function in the module: it resolves (a) whether a specific seat was requested or any free seat should be picked, (b) whether the target seat/table has room for the guest's whole party (guest + named `GuestParty` members), (c) seats party members via an outward search from the anchor seat (alternating +1/-1 seat-number offset, wrapping around the table) so a party is seated contiguously where possible, and (d) atomically clears any prior assignment for the guest and each party member before creating the new ones (required because `SeatingAssignment.guestId` and `PartySeatingAssignment.partyMemberId` are both @unique).
- Declining guests can still be assigned (with a warning, not a block) — a planner may want to seat someone who declined online but is expected to show up anyway. INFERRED rationale from the code comment in `capacity.ts`.
- `updateTable` capacity-shrink behaviour: removes the highest-seatNumber seats first; for each removed seat that was occupied (by a guest directly or by a party member), the **entire party** sharing that guest is unassigned (assignment rows deleted, not the seat/guest itself), and their names are returned to the caller for UI display.

5.6 Vendors module (`apps/api/src/modules/vendors`)

Files: `vendors.routes.ts`, `vendors.service.ts`, `vendors.schema.ts`.

- CRUD scoped to one event via `getOwnedEvent`. `cost` is accepted/returned in whole currency units by the API layer but stored as `costCents` (integer, `Math.round(dollars * 100)`).
- `getVendorSummary` returns both a single blended `totalCost` (kept “for backwards compatibility”, per the code comment) and `costsByCurrency` (an array of `{currency, total}`, sorted alphabetically by currency code) — the frontend Vendors tab and Analytics page display the per-currency breakdown, not the blended figure (see `06-ui-ux-specification.md`).
- Changing a vendor’s status field triggers an in-app notification to the event owner; changing any other field does not.

5.7 Products / Orders (Merchandise) module (`apps/api/src/modules/products`)

Files: `products.routes.ts`, `products.service.ts`, `products.schema.ts`, `orders.routes.ts`, `orders.controller.ts`, `orders.service.ts`, `orders.schema.ts`, `shop.routes.ts`, `webhook.routes.ts`.

- Planner-side CRUD (`products.routes.ts`, requires auth via the parent events router) vs. guest-facing public routes (`shop.routes.ts`, no auth, keyed by `rsvpToken`) are separate route files sharing `products.service.ts/orders.service.ts`.
- `listPublicProducts` returns `enabled: false` with an empty product list if `Event.merchandiseEnabled` is false, without leaking the event’s product catalogue. It also returns `paymentOptionsByCurrency` so the shop UI can disable currencies with no connected, ready payout account.
- Checkout (`createCheckoutSession`) validates: product existence/eventId match, active flag is implicit (only active products are listed publicly, but the service re-validates against `active: true` at checkout too — `CONFIRMED`, `products.findMany({..., active: true})` in `createCheckoutSession`), stock sufficiency, single-currency cart, and at least one connected+ready payout account for that currency. It then creates a `PENDING Order + OrderItems`, computes `platformFeeCents` from `env.platformFeePercent`, and dispatches to one of three provider-specific checkout starters based on the chosen (or preferred-default) provider. If the provider call throws, the `PENDING` order is deleted (no orphaned rows).
- Stripe checkout uses Stripe Checkout Sessions in mode: “payment” with a Connect **destination charge**: `application_fee_amount` (our cut) + `transfer_data.destination` (planner’s connected account) on the `payment_intent_data`.
- Paystack checkout uses `/transaction/initialize` with the amount in the smallest currency unit (kobo, matching our own cents convention 1:1) and a subaccount code for the split (percentage fixed at subaccount-creation time, not re-specified per transaction).
- PayPal checkout creates a PayPal Order routed via `payee.email_address`; it first attempts to attach `payment_instruction.platform_fees` (our cut) and, if PayPal rejects that (no Partner/BN-code enrollment), silently retries without a fee so checkout still completes — the order’s `platformFeeCents` is set to 0 in that fallback

case, not the originally computed value, so the on-file fee always matches what was actually collected.

- Webhooks (`webhook.routes.ts`) are mounted in `app.ts` **before** the global `express.json()` parser, using `express.raw()` for exactly these two paths, so Stripe/Paystack signature verification sees the untouched raw bytes.
- `finalizeOrderPaid` is the single shared “mark PAID” transition for both merchandise and ticket orders (differentiated by `Order.kind`): for TICKET orders it creates one Ticket row per unit with a random code; for MERCHANDISE orders it decrements `Product.stockQuantity` (only for products with a non-null stock quantity — unlimited-stock products are left untouched).
- Every payment provider event (success, decline, expiry) — not just ones ending in a paid order — is written to `PaymentEvent` via a best-effort `logPaymentEvent` helper that swallows its own errors so a logging failure never breaks the payment flow itself.

5.8 Payouts module (`apps/api/src/modules/payouts`)

Files: `payouts.routes.ts`, `payouts.service.ts`, `payouts.schema.ts`.

- `isPayoutAccountConnected` is the single source of truth for “ready to accept payments”: Stripe Connect requires `stripeOnboardingComplete = true`, Paystack requires a saved `paystackSubaccountCode`, PayPal requires a saved `paypalEmail`.
- `listPayoutAccounts` actively re-syncs any Stripe account still marked incomplete by calling Stripe’s Accounts API directly (`charges_enabled` & `payouts_enabled`) — a belt-and-suspenders fallback for missed/delayed `account.updated` webhooks, run on every list-fetch for incomplete Stripe accounts specifically (not for already-complete ones).
- Stripe Connect: creates an Express account (`business_type: "individual"`, capabilities: `{card_payments, transfers}`) scoped to US (USD) or GB (GBP) — no other countries are supported, since NGN routes through Paystack instead. Returns a fresh, single-use Account Link to Stripe’s hosted onboarding UI; safe to call again for an incomplete account (a fresh link is issued each time).
- Paystack: resolves the account-number/bank-code pair against Paystack’s `/bank/resolve` endpoint before creating anything, then creates a Subaccount with `percentage_charge` fixed to the *current* `env.platformFeePercent` at creation time — a later change to that env var does not retroactively change already-connected subaccounts’ split. The raw account number is sent to Paystack in this one request and never persisted; only the resulting `subaccount_code` and a masked last-4 are stored.
- PayPal: no hosted onboarding — just saves the planner’s PayPal email as the payee for `payee.email_address`-routed orders.
- `connectStripe`, `connectPaystack`, and `connectPaypal` all call `getOwnedEventStrict` (no admin bypass) — an admin browsing a subscriber’s event in support mode cannot create or modify that subscriber’s payout destinations.

5.9 Tickets module (apps/api/src/modules/tickets)

Files: ticketTypes.routes.ts, ticketTypes.controller.ts, ticketTypes.service.ts, ticketTypes.schema.ts, checkout.routes.ts, checkout.controller.ts, checkout.service.ts, checkout.schema.ts.

- Ticket types are scoped per-event, ordered by an explicit sortOrder (drag-reorderable via POST /reorder, which verifies every submitted id belongs to the event before touching anything).
- deleteTicketType is blocked once quantitySold > 0 — the planner must set the type inactive (isActive: false) instead of deleting a type with sales history, preserving OrderItem/Ticket referential integrity.
- Capacity reservation (reserveTickets) uses a single conditional UPDATE ... SET "quantitySold" = "quantitySold" + n WHERE "isActive" = true AND ("quantityTotal" IS NULL OR "quantityTotal" - "quantitySold" >= n) raw SQL statement rather than a read-then-write pair — this is what makes concurrent checkouts for the last remaining ticket race-safe at the database level, not just at the application level.
- Public checkout (createTicketCheckoutSession) re-validates on-sale window (salesStartAt/salesEndAt), min/max-per-order bounds, remaining capacity, and single-currency-cart, in addition to reserving capacity — all inside one \$transaction, so a failed reservation rolls back the whole attempted order creation.
- Ticket checkout reuses the exact same startStripeCheckout/startPystackCheckout/startPaypalCheckout/capturePaypalOrderCore functions from orders.service.ts (imported directly, not duplicated) — merchandise and ticket checkout are two callers of one shared payment-provider integration layer.
- Door check-in (checkInTicketByCode) is a separate credential/flow from guest check-in (checkInGuestByToken in the guests module) — ticket buyers have no Guest row at all, so there is nothing to key a guest-list lookup off.

5.10 Notifications module (apps/api/src/modules/notifications)

- Notifications are created as a direct side effect inside the service function that causes them (RSVP change, vendor status change, order paid) — there is no generic event bus/queue. INFERRED design rationale from the code comment: “keeping generation as small, targeted helpers ... matching the rest of the codebase’s direct-call style,” explicitly chosen over a more generic mechanism at the app’s current size.
- listNotifications supports an unreadOnly filter and returns both the page of notifications and a separate always-fresh unread count.

5.11 Insights module (apps/api/src/modules/insights)

- Computed on every request from live guest/event data — no Insight table exists; an insight simply stops being returned once its underlying condition resolves.
- Insight types generated: RSVP deadline within 7 days, unassigned VIP guests (individually, up to 5, then rolled into a summary count), unassigned non-VIP

confirmed guests (rolled into one summary insight), missing meal selections (only checked if allowMealSelection is on for that event), and outstanding (pending/maybe) responses. Each is tagged ACTION_REQUIRED or UPDATE severity and sorted with ACTION_REQUIRED first.

- Can be scoped to a single event or run across all of a planner's events (GET /api/insights?eventId=... optional).

5.12 Analytics module (apps/api/src/modules/analytics) and Admin analytics

- Planner analytics (getAnalyticsOverview) is entirely derived from current Event/Guest/Vendor rows — there is no historical snapshot table, so there are no true trend lines over time at the per-planner level (only the platform-wide admin view has a day-by-day trend, see below).
- Platform analytics (admin.service.ts getPlatformAnalytics) additionally computes a 30-day signup/event-creation trend via a raw SQL query using Postgres generate_series LEFT JOINed against per-day counts, so days with zero activity still appear as zero rather than being absent from the series.
- Revenue is always reported as an array grouped by (currency, provider), never summed into one number — the code comment explicitly states this is deliberate (“summing cents across USD/GBP/NGN would be meaningless”).

5.13 Admin module (apps/api/src/modules/admin)

- listAllUsers/listAllEvents are cross-subscriber views with no owner-scoped equivalent (a planner never sees another planner's list) — these exist only under /api/admin.
- getAuditLog/getPaymentEvents support optional filters (eventId, adminUserId / eventId, orderId, status, provider) and a limit.
- Admin event drill-in deliberately has **no** separate /api/admin/events/:id/* sub-API — it reuses /api/events/:eventId/* end to end via the ownership-bypass mechanism described in §5.2.

5.14 Articles & Landing modules

- Articles: admin-only CRUD + publish/unpublish workflow (DRAFT/PUBLISHED via ArticleStatus), auto-generated immutable slug (same collision-suffix pattern as event publicSlug), optional cover image (bytes-in-Postgres). Public reads only ever return status: PUBLISHED articles, ordered by publishedAt (not createdAt) so republishing an old draft brings it back to the top of the list.
- Landing services (“what we offer” cards): admin-only CRUD + reorder (sortOrder), public read returns only isActive: true rows ordered by sortOrder. icon is a string constrained to a fixed enum on the frontend (SERVICE_ICON_OPTIONS in apps/web/src/types/index.ts), not free-text/SVG — explicitly to avoid it becoming an injection vector (code comment in landing.schema.ts, INFERRED to be an XSS-prevention rationale from the comment text “not free-text/SVG upload, to keep this from becoming an XSS vector”).

06 — UI/UX Specification

Screen inventory is CONFIRMED against `apps/web/src/App.tsx` (route table) and the corresponding page component files (all confirmed to exist via `apps/web/src/pages/**/*.*.tsx`). Field-level and interaction-level detail for each screen reflects direct code review performed across this documentation effort and prior work in this project; where a specific screen's internals were not re-verified in this pass, that is marked INFERRED (from component/hook naming) rather than presented as directly observed.

6.1 Navigation shell

`apps/web/src/components/layout/DashboardLayout.tsx` (CONFIRMED, full file reviewed) is the single shared shell for every authenticated route. It renders a top navigation bar (not a sidebar — the code comment notes this replaced an earlier left-sidebar design) that switches content based on context:

- **Outside an event** (“global” mode): “My Events” and “Analytics” links, plus “Admin” if `user.role === "ADMIN"`.
- **Inside an event** (`/events/:eventId/...`): eight tabs — Overview, Guests, RSVP, Vendors, Merchandise, Tickets, Seating, Check-in — each with an icon and a hover/focus tooltip (via the shared Tooltip component) describing its purpose.

When an admin has drilled into a subscriber’s event they don’t own (`event.userId !== user.id`), a “Support view — editing as admin” badge appears in the event sub-header, and the delete-event button is hidden (deleting is owner-only server-side; hiding it client-side avoids a confusing failed request). A mobile hamburger menu collapses the same nav for small screens.

Global chrome present on every authenticated screen: a notification bell (`NotificationBell.tsx`, shows unread count, dropdown list, “mark all read”) and a user menu (`UserMenu.tsx`, avatar with initials + online-style dot, dropdown with “Log out” — which navigates to `/`, the landing page, per the current implementation).

6.2 Screen inventory

Route	Component	Auth	Purpose
<code>/</code>	<code>pages/marketing/LandingPage.tsx</code>	Public	Marketing homepage: hero, services grid (“How it works”), payments callout, FAQ accordion, articles teaser, CTA banner

/articles	pages/marketing/ArticlesListPage.tsx	Public	Published blog article listing
/articles/:slug	pages/marketing/ArticleDetailPage.tsx	Public	Single article view
/login	pages/auth/LoginPage.tsx	Public	Email/password login
/register	pages/auth/RegisterPage.tsx	Public	Account creation
/rsvp/:token	pages/rsvp/PublicRsvpPage.tsx	Public (token)	Shared-link RSVP form + shop (ShopSection.tsx)
/rsvp/invite/:invitationToken	pages/rsvp/PublicRsvpPage.tsx	Public (token)	Personalised, pre-filled RSVP form
/tickets/:slug	pages/tickets/PublicTicketEventPage.tsx	Public (slug)	Public ticket listing + checkout
/events	pages/events/EventsListPage.tsx	Auth	“My Events” dashboard — event cards with RSVP/table progress
/events/:eventId	(redirect)	Auth	Redirects to .../overview
/events/:eventId/overview	pages/events/OverviewTab.tsx (via EventTabPage.tsx)	Auth	Per-event readiness dashboard, quick-access panel, stat cards
/events/:eventId/guests	pages/events/GuestsTab.tsx	Auth	Guest list, search/filter, CSV import/export, PDF export
/events/:eventId/rsvp	pages/events/RsvpTab.tsx	Auth	RSVP funnel dashboard, invite sending, invitation card upload
/events/:eventId/vendors	pages/events/VendorsTab.tsx	Auth	Vendor CRUD, per-currency cost totals
/events/:eventId/merchandise	pages/events/MerchandiseTab.tsx	Auth	Product CRUD, order summary, payouts settings

/events/:eventId/ tickets	pages/events/ TicketsTab.tsx	Auth	entry point Ticket type CRUD, publish/public- listing settings (PublishSettingsPa nel.tsx)
/events/:eventId/ seating	pages/events/ seating/ SeatingTab.tsx	Auth	Visual drag-and- drop seating canvas (SeatingCanvas.ts x, Konva-based), guest sidebar, selection panel
/events/:eventId/ checkin	pages/events/ CheckInTab.tsx	Auth	Manual + QR-scan (TicketScanPanel.t sx) check-in kiosk view
/analytics	pages/analytics/ AnalyticsPage.tsx	Auth	Cross-event analytics for the logged-in planner
/admin	pages/admin/ AdminPage.tsx (+ PlatformAnalytics Tab.tsx, ArticlesTab.tsx, ServicesTab.tsx)	Auth + ADMIN	Subscriber/event browser, audit log, payment log, platform analytics, articles/services management
* (unmatched)	(<i>redirect</i>)	—	Redirects to /events

CONFIRMED — route table in apps/web/src/App.tsx; component file existence via Glob of apps/web/src/pages/**/*.*.tsx.

6.3 Route protection

- ProtectedRoute (apps/web/src/components/ProtectedRoute.tsx): while AuthContext’s isLoading is true, renders a spinner; once resolved, redirects to /login (preserving the attempted location in router state) if no user is present, otherwise renders the nested route via <Outlet />.
- AdminRoute: same pattern, but redirects non-ADMIN users to /events rather than /login — this is a client-side convenience only; the API independently enforces requireAdmin on every /api/admin/* call regardless of what the UI shows.

6.4 Design system

CONFIRMED from apps/web/tailwind.config.js and this project’s own prior styling work: a “duotone” palette with brand (violet/purple, primary — buttons, links, active nav state) and coral (secondary accent — used for select stat cards, scarcity badges, and alternating

icon tiles across the planner dashboard, Overview, Analytics, Vendors, and Merchandise screens specifically, per this session's earlier UI work) plus a retinted slate neutral scale and semantic success/warning/danger/info ramps. Fonts: DM Sans (body) and Outfit (display), loaded via @fontsource/* packages (CONFIRMED, apps/web/package.json). Deliberately **not** applied to Guests/RSVP/Seating/Check-in status indicators, which rely on semantic (not brand) colour to convey RSVP/check-in state — an explicit scoping decision made during this project's styling work, not a limitation discovered by omission.

Shared UI primitives (CONFIRMED file existence, apps/web/src/components/ui/*): Button, Badge, Card/StatCard, Modal, Input, Tooltip, Spinner, EmptyState, Avatar, ProgressBar, Stepper, DonutChart, RadialProgress, TrendChart, CategoryChart, ChartTypeToggle, ExportMenu, QrScanner, AuthedImage.

6.5 Key interaction patterns

- **Tooltips on hover/focus:** nav tabs and icon-only buttons (notification bell, remove/edit icon buttons on Vendors/Tickets/Merchandise) show a descriptive label on hover or keyboard focus via the shared Tooltip component (CSS-only, group/tooltip pattern — CONFIRMED, apps/web/src/components/ui/Tooltip.tsx).
- **Multi-currency totals:** any stat card summing money across records that may span currencies (Vendors “Total Cost”, Analytics “Vendor Spend”) renders one line per currency with its own symbol, rather than a single figure with a fixed \$ icon — CONFIRMED, apps/web/src/lib/format.ts (formatMoneyBreakdownSymbols/formatMoneyBreakdownParts), applied in VendorsTab.tsx and AnalyticsPage.tsx.
- **Empty/loading/error states:** EmptyState and Spinner components exist and are used across list-type screens (INFERRED as a general pattern from component naming and prior-session UI polish work covering this explicitly — see Known Issues for any specific screen where this wasn't verified this pass).
- **Toasts:** sonner (CONFIRMED, apps/web/package.json) is used for transient success/error feedback (e.g. “Event deleted”) — seen directly in DashboardLayout.tsx's delete-event handler.
- **PWA install prompt:** InstallPrompt.tsx, rendered at the app root outside the route tree, listens for the browser's install-eligibility signal and offers a banner (INFERRED standard beforeinstallprompt pattern from the component's existence and mount location; internals not re-read this pass).

6.6 User flow diagrams

Guest RSVP flow

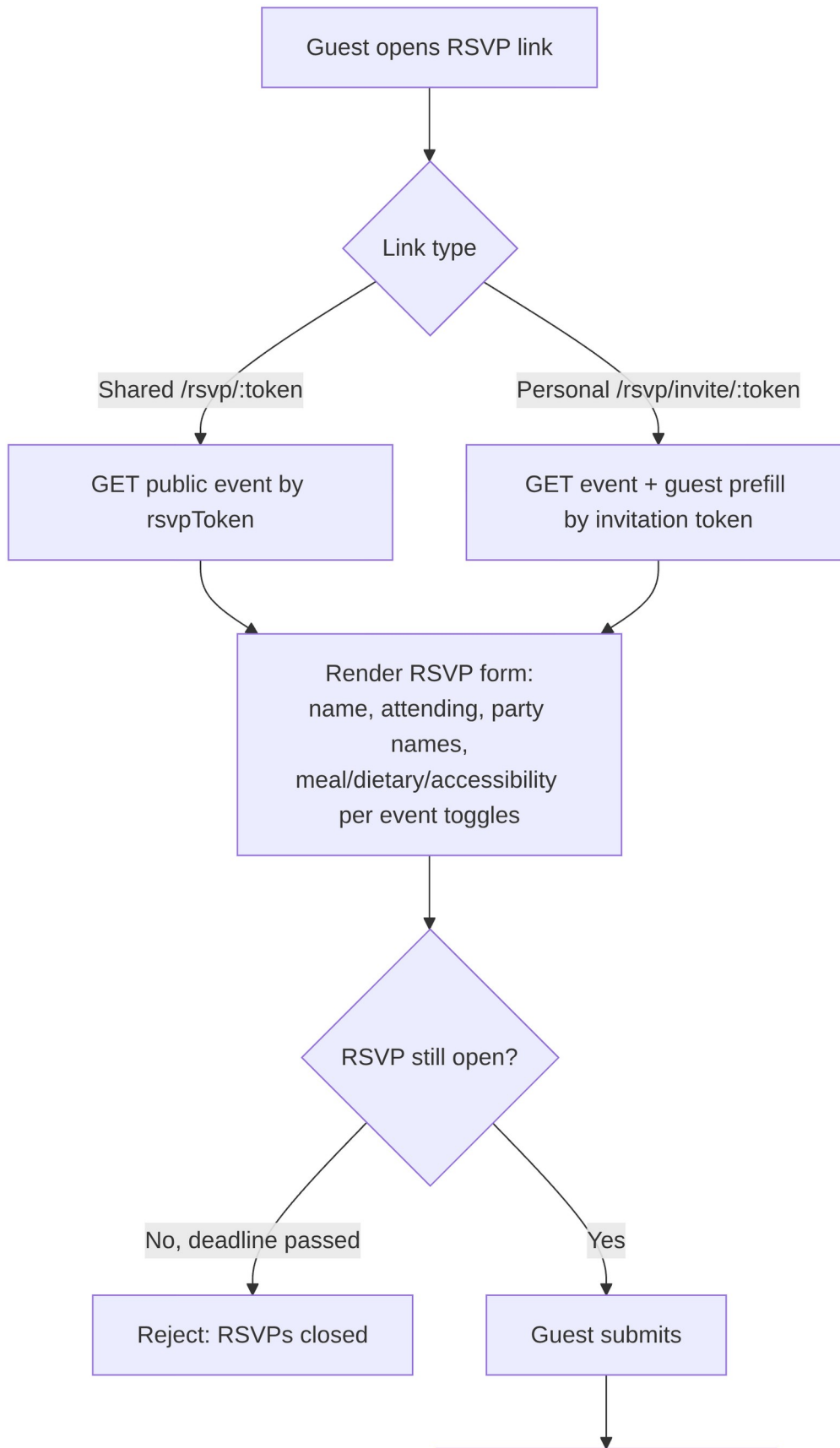


Diagram 1 — 06-ui-ux-specification

CONFIRMED against apps/api/src/modules/rsvp/rsvp.service.ts,
apps/api/src/modules/notifications/notifications.service.ts.

Ticket purchase and door check-in flow

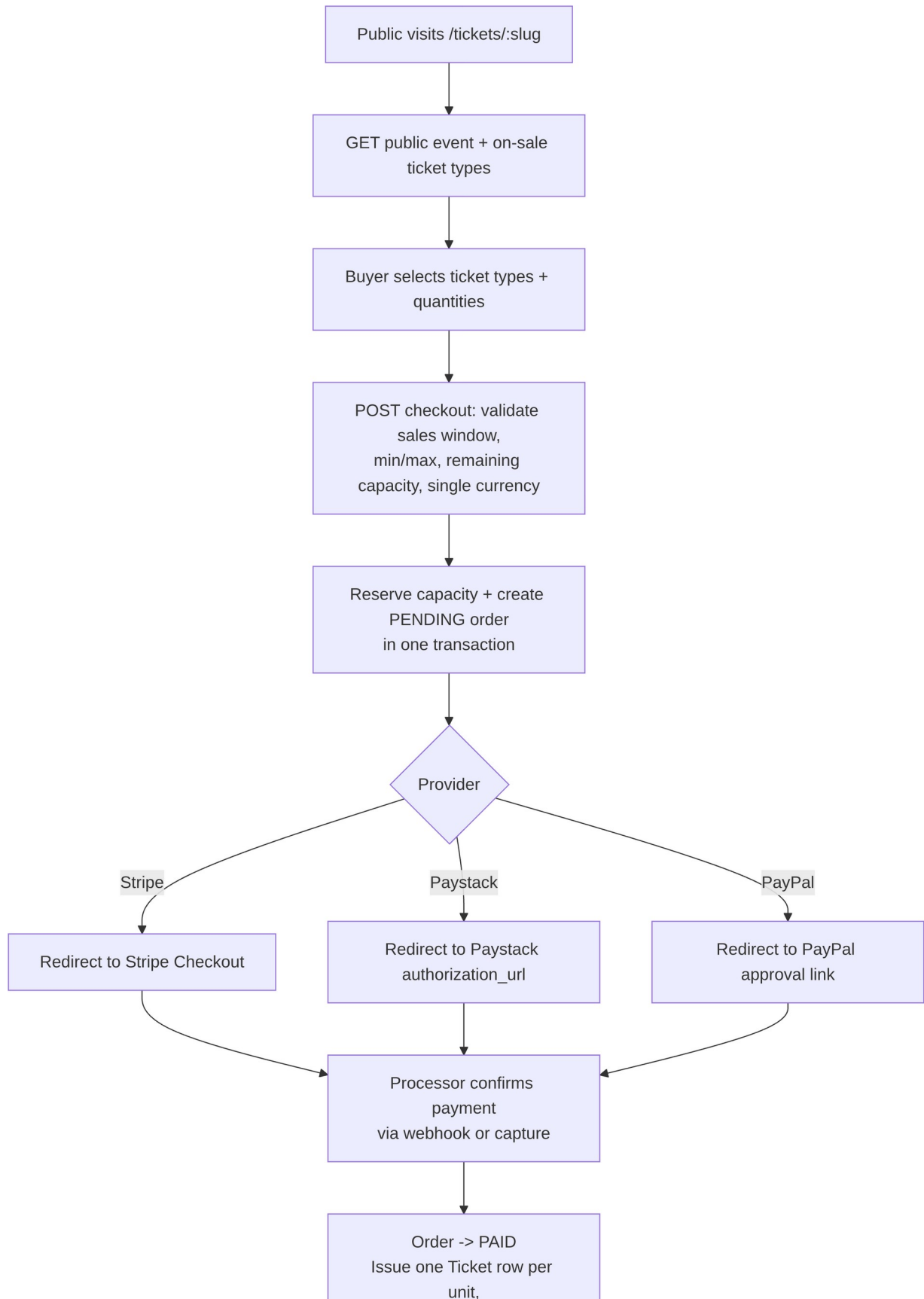


Diagram 2 — 06-ui-ux-specification

CONFIRMED against apps/api/src/modules/tickets/checkout.service.ts, ticketTypes.service.ts, apps/api/src/modules/products/orders.service.ts.

6.7 Screens not independently verified this pass

Field-by-field form contents of EventFormModal, GuestFormModal, CsvImportModal, InviteModal, VendorFormModal, ProductFormModal, TicketTypeFormModal, ServiceFormModal, and ArticleFormModal were not re-read line-by-line during this documentation pass (their existence and general purpose are CONFIRMED by filename and route wiring; their exact field list is INFERRED from the corresponding backend Zod schema, which does constrain what a well-behaved form must submit). Treat exact on-screen field order/labels as INFERRED, not CONFIRMED, until cross-checked directly against these component files.

07 — System Architecture

7.1 High-level architecture

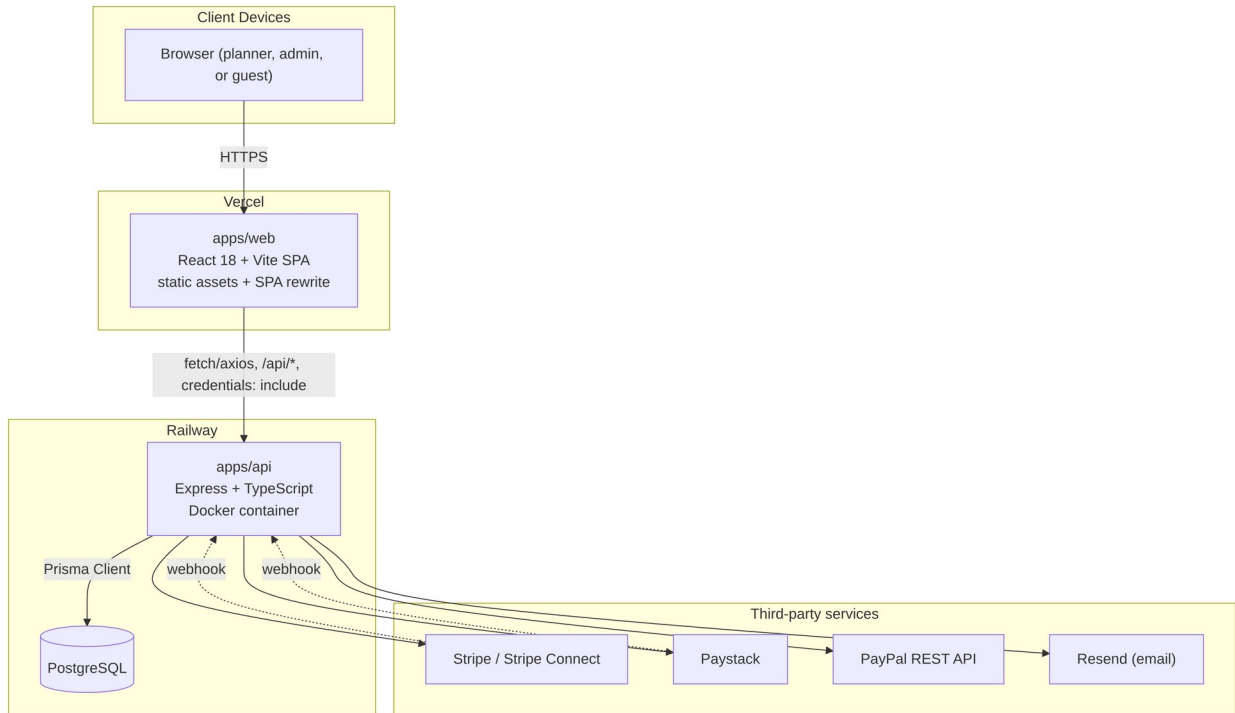


Diagram 1 — 07-system-architecture

CONFIRMED — apps/api/railway.json (Dockerfile build, prisma migrate deploy on boot, /health healthcheck), apps/web/vercel.json (npm run build, dist output, SPA rewrite to index.html), apps/api/src/lib/prisma.ts, apps/api/src/lib/stripeClient.ts/paystackClient.ts/paypalClient.ts, apps/api/src/modules/guests/invite.service.ts (Resend).

7.2 Component / module architecture (backend)

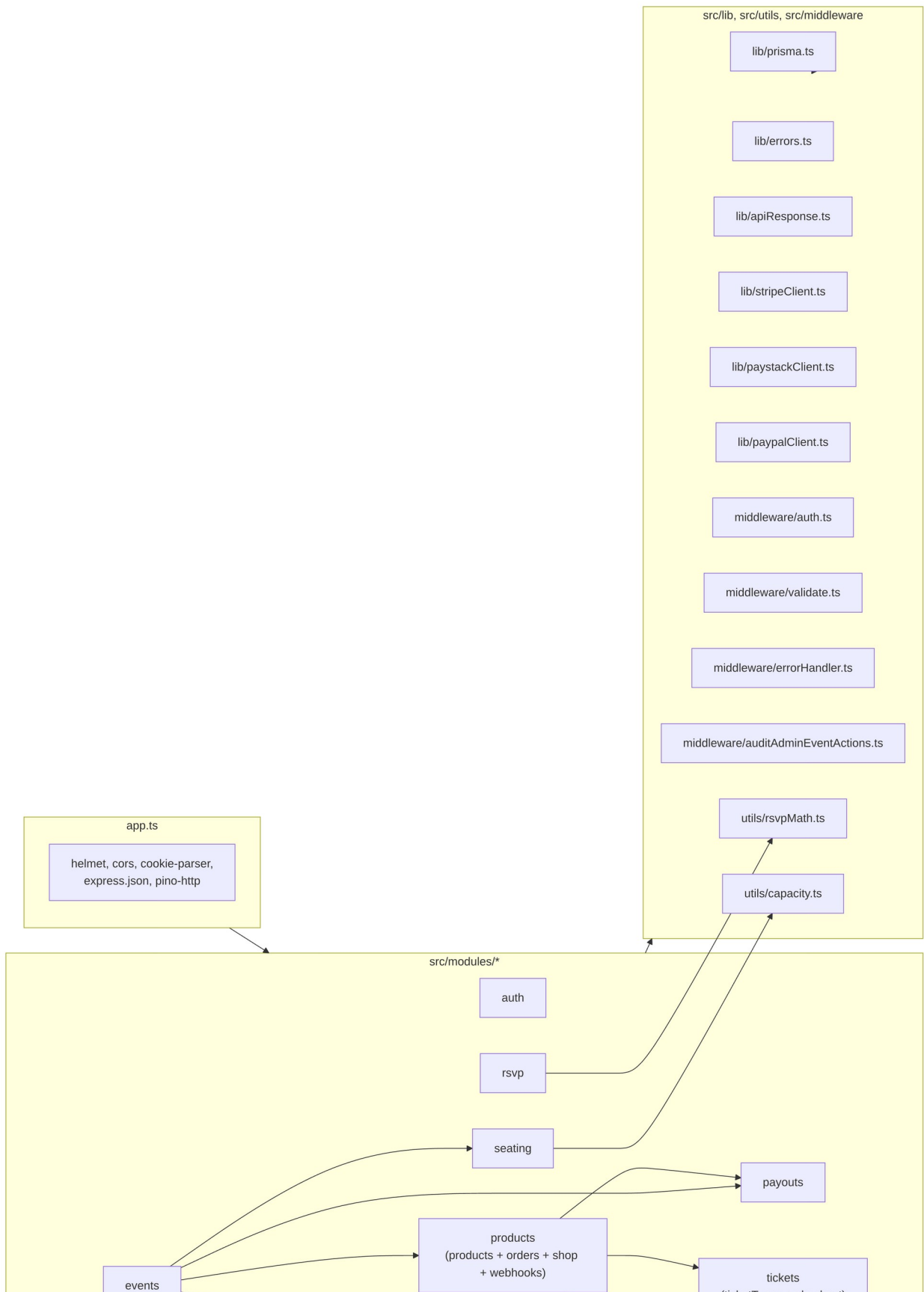


Diagram 2 — 07-system-architecture

CONFIRMED — direct import graph observed across `apps/api/src/app.ts` and each module's `*.routes.ts/*.service.ts` (e.g. `events.routes.ts` mounting `guestsRouter/seatingRouter/vendorsRouter/productsRouter/ordersRouter/payoutsRouter/ticketTypesRouter`; `tickets/checkout.service.ts` importing `startStripeCheckout` etc. directly from `products/orders.service.ts`; `rsvp.service.ts` calling `notifications.service.ts`).

7.3 Request / data flow (typical authenticated write)

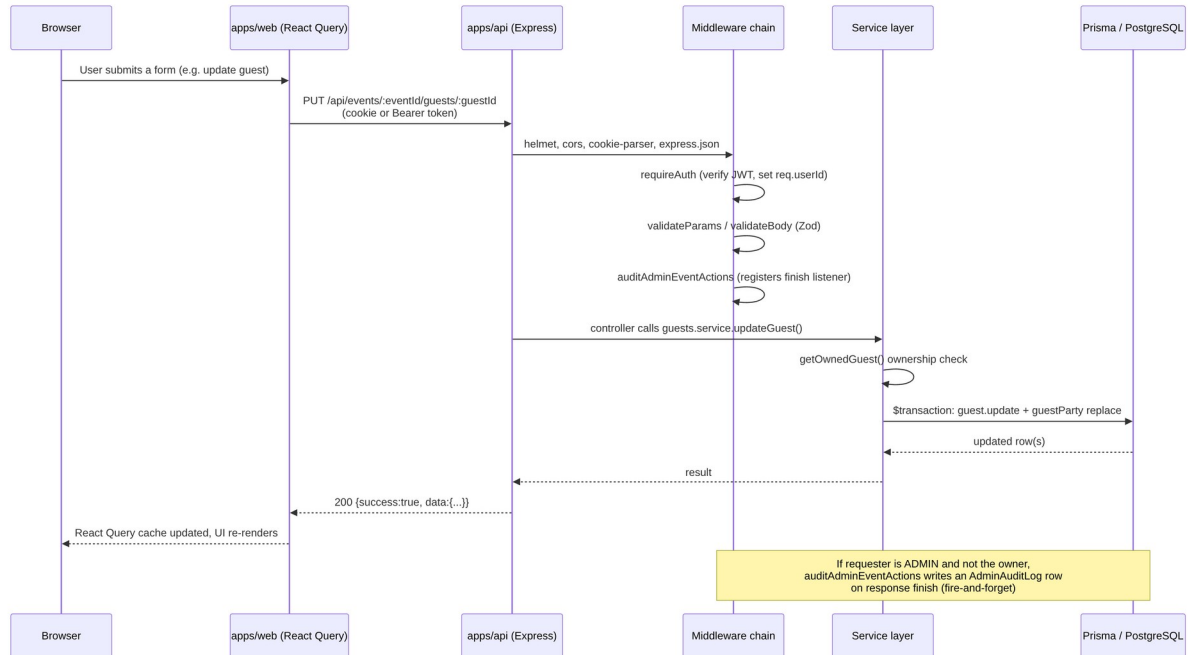


Diagram 3 — 07-system-architecture

CONFIRMED — `apps/api/src/app.ts` middleware order, `apps/api/src/middleware/*`, `apps/api/src/modules/guests/guests.service.ts` (`updateGuest`), `apps/api/src/middleware/auditAdminEventActions.ts`. React Query usage on the frontend is CONFIRMED by `apps/web/package.json` (`@tanstack/react-query`) and the `hooks/use*.ts` file set; exact cache-invalidation behaviour per hook is INFERRED (standard React Query pattern), not re-verified hook-by-hook this pass.

7.4 Authentication flow

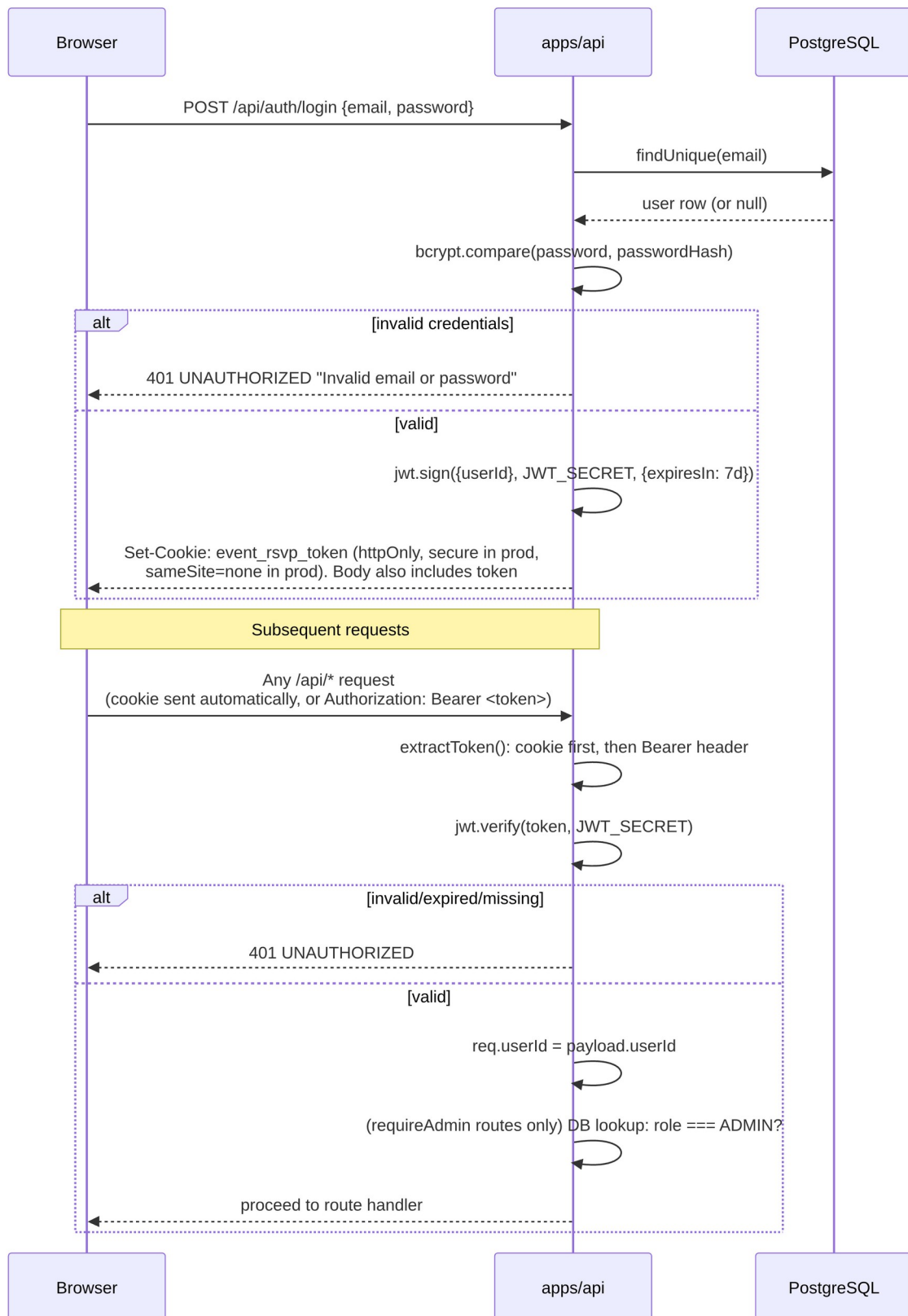


Diagram 4 — 07-system-architecture

CONFIRMED — apps/api/src/modules/auth/auth.controller.ts, auth.service.ts,
apps/api/src/middleware/auth.ts, apps/api/src/utils/jwt.ts,
apps/api/src/utils/password.ts.

7.5 Major business workflow — multi-processor checkout routing

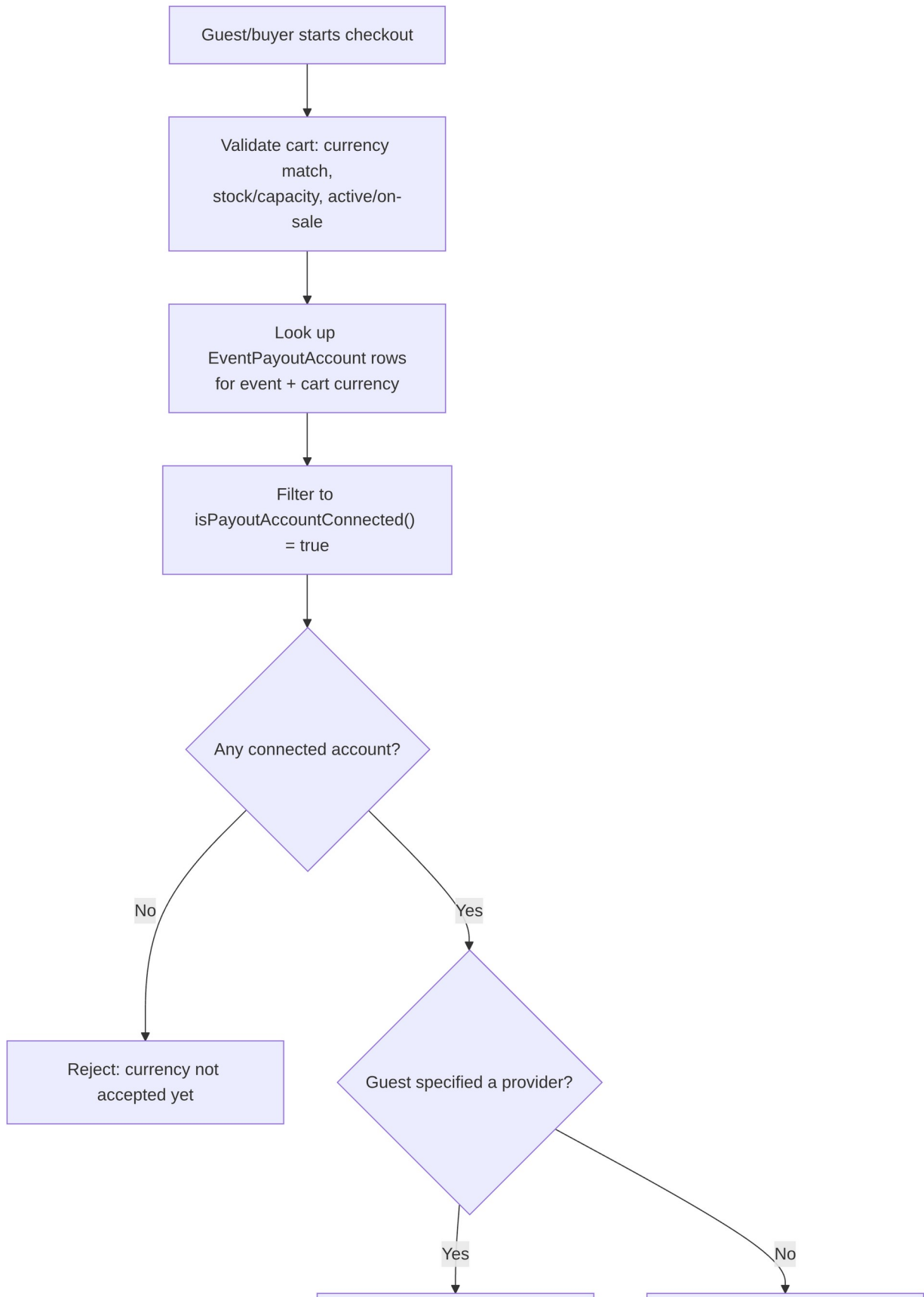


Diagram 5 — 07-system-architecture

CONFIRMED — apps/api/src/modules/products/orders.service.ts (createCheckoutSession, DEFAULT_PROVIDER_PREFERENCE, finalizeOrderPaid), apps/api/src/modules/tickets/checkout.service.ts (createTicketCheckoutSession, reusing the same starters). This single flow serves both the merchandise shop and public ticket sales, differentiated only by Order.kind.

7.6 Deployment topology

- apps/api builds via a Dockerfile on Railway; the container's start command runs prisma migrate deploy (25-second timeout, logged but not fatal if it times out — the server starts regardless) then node dist/index.js; Railway health-checks GET /health (30s timeout, restarts on failure up to 3 times). CONFIRMED — apps/api/railway.json.
- apps/web builds via npm run build (tsc + vite build) on Vercel, serving the dist directory with a catch-all rewrite to index.html for client-side routing. CONFIRMED — apps/web/vercel.json.
- Local development uses a single docker-compose.yml Postgres 16-alpine service; no other services (Redis, queues, etc.) are present in local dev tooling. CONFIRMED — docker-compose.yml.

See 15-deployment-and-infrastructure.md for full detail.

08 — Database Documentation

Source of truth: apps/api/prisma/schema.prisma (938 lines, PostgreSQL provider, Prisma Client 5). All 27 models below are CONFIRMED directly from this file. Field types shown are the Prisma schema types (not the raw Postgres column types, which Prisma derives automatically).

8.1 Entity-Relationship Diagram

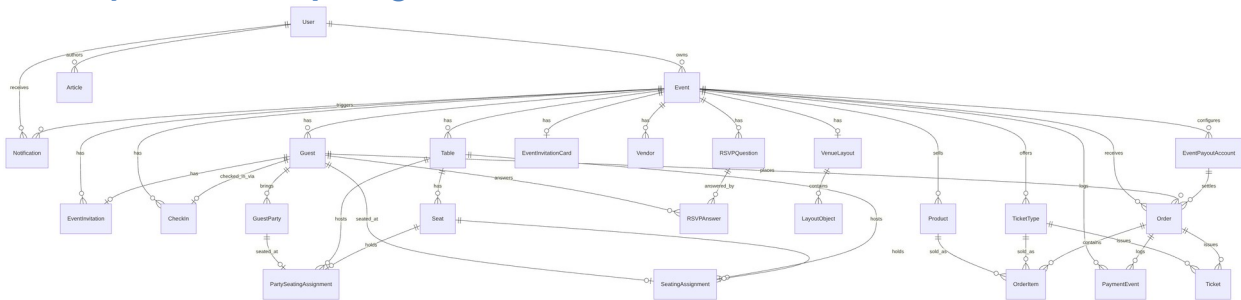


Diagram 1 — 08-database-documentation

CONFIRMED — all relations traced directly from @relation fields in apps/api/prisma/schema.prisma.

8.2 Model reference

For every model: field name, Prisma type, notes, Evidence (always apps/api/prisma/schema.prisma, model name given for lookup).

User (users)

Field

id
email
passwordHash
name
role
createdAt / updatedAt

Event (events)

Field	Type	Notes
id	String @id	cuid()
userId	String, FK → User, onDelete Cascade	
name, type, description,	mixed	type default OTHER

date, startTime, endTime, venueName, venueAddress, capacity, imageUrl		
rsvpToken	String @unique @default(cuid())	bearer credential for the public RSVP page
rsvpOpen	Boolean @default(true)	
rsvpDeadline	DateTime?	
customMessage	String?	
isPublic	Boolean @default(false)	switches event into public- ticketing mode
publicCategory	PublicEventCategory?	NIGHTLIFE \\ BOAT_CRUISE \\ CONCERT \\\ FESTIVAL \\ COMEDY_SHOW \\ PRIVATE_PARTY \\ OTHER
publicSlug	String? @unique	generated once, immutable
publicDescription, minAge	mixed	
coverImageData / coverImageMimeType	Bytes? / String?	image stored in Postgres
allowPlusOnes, allowPlusOneNames, allowMealSelection, allowDietary, allowAccessibilityInfo, allowSpecialRequests	Boolean, all @default(true)	RSVP form feature toggles
merchandiseEnabled	Boolean @default(false)	
createdAt / updatedAt	DateTime	
Index	@@index([userId])	

Guest (guests)

Field	Type	Notes
id, eventId (FK, Cascade)		
firstName, lastName, email?, phone?, groupName?		
rsvpStatus	RsvpStatus @default(PENDING)	PENDING \\ CONFIRMED \\\

		DECLINED \ MAYBE
rsvpRespondedAt	DateTime?	
additionalGuestsCount	Int @default(0)	derived from party list length when named party members are used
mealPreference, dietaryRequirements, accessibilityRequirements, specialNotes	String?	
isVip	Boolean @default(false)	
checkedIn / checkedInAt	Boolean / DateTime?	
createdAt / updatedAt		
Indexes	[eventId], [eventId, rsvpStatus], [eventId, email]	

GuestParty (guest_party_members)

Field

id, guestId (FK, Cascade)

fullName

mealPreference, dietaryRequirements

createdAt

Index

EventInvitationCard (event_invitation_cards)

Field

id, eventId (FK, Cascade, @unique — one per event)

data

contentType, fileName, size

createdAt / updatedAt

EventInvitation (event_invitations)

Field

Type

Notes

id, eventId (FK, Cascade)

guestId String? @unique, FK

1:1 optional — one

	Cascade	invitation per guest
token	String @default(cuid()) @unique	personalised RSVP bearer credential
channel	String?	free text: “email sms whatsapp manual” per comment
sentAt	DateTime?	
createdAt		
Index	[eventId]	

RSVPQuestion (rsvp_questions) / RSVPAnswer (rsvp_answers)

Custom-question scaffolding. RSVPQuestion: label, type (default "text"), options (JSON-encoded string), required, order. RSVPAnswer: guestId + questionId (unique together), answerText. INFERRED status: no route file among the 22 confirmed route modules exposes CRUD for RSVPQuestion/RSVPAnswer — see 24-documentation-gap-analysis.md and 20-known-issues-risks-technical-debt.md; these two models appear to be schema scaffolding for a feature not yet wired to any API endpoint.

VenueLayout (venue_layouts) / LayoutObject (layout_objects)

VenueLayout: one per event (eventId @unique), name (default “Main Layout”), canvasWidth/canvasHeight (1600×1000 default), gridSize (20), backgroundColor (#f8fafc). LayoutObject: type (LayoutObjectType enum: STAGE, DANCE_FLOOR, BAR, BUFFET, ENTRANCE, EXIT, TOILETS, DJ_BOOTH, VIP_AREA, CUSTOM), label?, x/y/width/height/rotation (Float), color?.

Table (tables) / Seat (seats)

Table: eventId (FK Cascade), name, shape (TableShape: ROUND, SQUARE, RECTANGLE, OVAL, BANQUET, HEAD, CUSTOM; default ROUND), capacity (default 8), x/y/width/height/rotation. Seat: tableId (FK Cascade), seatNumber, x/y; @@unique([tableId, seatNumber]).

SeatingAssignment (seating_assignments) / PartySeatingAssignment (party_seating_assignments)

Both: guestId/partyMemberId respectively as @unique (one active assignment per guest/party-member), tableId (FK Cascade), seatId? (FK, onDelete: SetNull — deleting a Seat does not delete the assignment row, it just clears seatId, which is why seating.service.ts explicitly deletes assignment rows outright when shrinking a table rather than relying on cascade), assignedAt.

CheckIn (check_ins)

eventId (FK Cascade), guestId (@unique — one check-in record per guest, upserted), checkedInAt, checkedInBy? (free-text staff identifier).

Vendor (vendors)

eventId (FK Cascade), name, category (VendorCategory: CATERING, VENUE, PHOTOGRAPHY, VIDEOGRAPHY, FLORAL, MUSIC_ENTERTAINMENT, DECOR, RENTALS, TRANSPORTATION, BEAUTY, STATIONERY, OTHER; default OTHER), status (VendorStatus: CONTACTED, QUOTE_RECEIVED, BOOKED, CONFIRMED, CANCELLED; default CONTACTED), contactName?/email?/phone?/website?, costCents? (minor units), currency (Currency enum: USD, GBP, NGN; default USD), depositPaid (default false), notes?.

Notification (notifications)

userId (FK Cascade), eventId? (FK Cascade), type (NotificationType: RSVP_CONFIRMED, RSVP_DECLINED, VENDOR_STATUS_CHANGED, ORDER_PAID, SYSTEM), title, body?, link? (frontend deep-link path), read (default false), readAt?. Indexes: [userId, read], [eventId].

Product (products)

eventId (FK Cascade), name, description?, priceCents (minor units), currency (default USD), stockQuantity? (null = unlimited), active (default true — hides without deleting order history), imageData?/imageMimeType? (bytes in Postgres).

EventPayoutAccount (event_payout_accounts)

eventId (FK Cascade), currency, provider (PayoutProvider: STRIPE_CONNECT, PAYSTACK, PAYPAL). Stripe fields: stripeAccountId?, stripeOnboardingComplete (default false). Paystack fields: paystackSubaccountCode?, paystackBankName?, paystackAccountLast4?. PayPal field: paypalEmail?. @@unique([eventId, currency, provider]) — a planner may connect more than one provider per currency, guest picks at checkout. **No raw bank account numbers are ever stored** (per code comment and confirmed against payouts.service.ts — only Paystack's resulting subaccount code and a masked last-4 are persisted).

TicketType (ticket_types)

eventId (FK Cascade), name, description?, priceCents, currency, quantityTotal? (null = unlimited), quantitySold (default 0, denormalized counter, incremented only inside the reservation transaction — see ticketTypes.service.ts reserveTickets), salesStartAt?/salesEndAt?, minPerOrder (default 1), maxPerOrder (default 10), sortOrder (default 0), isActive (default true).

Ticket (tickets)

ticketTypeId (FK Cascade), orderId (FK Cascade), code (String @unique @default(cuid()) — the QR payload, deliberately a separate random token from the row id so codes can't be enumerated), status (TicketStatus: VALID, CHECKED_IN, CANCELLED; default VALID), attendeeName?/attendeeEmail?, checkedInAt?/checkedInBy?.

AdminAuditLog (admin_audit_logs)

adminUserId, adminEmail (captured directly, not just FK'd — survives account deletion), eventId?/eventName? (same "captured directly" pattern — survives event deletion), method, summary (human-readable, generated by auditAdminEventActions.ts's describe()). **Deliberately has no FK/cascade to Event or User** per code comment — this history must survive even if the referenced event or admin account is later deleted.

PaymentEvent (payment_events)

eventId? (FK, onDelete: SetNull), orderId? (FK, onDelete: SetNull), provider?, type (raw provider event type string, e.g. "checkout.session.completed"), status (PaymentEventStatus: SUCCESS, FAILED, EXPIRED, INFO), amountCents?, currency?, message?, rawPayload (String — JSON.stringify of the provider's object, truncated to 20,000 characters per orders.service.ts).

Order (orders) / OrderItem (order_items)

Order: eventId (FK Cascade), guestId? (FK, onDelete: SetNull), kind (OrderKind: MERCHANDISE, TICKET; default MERCHANDISE — this single field is what lets one Order/OrderItem model serve two different product types), guestName/guestEmail (captured directly, not solely via guestId), status (OrderStatus: PENDING, PAID, CANCELLED; default PENDING), currency, provider?, payoutAccountId? (FK, SetNull), totalCents, platformFeeCents? (frozen at checkout time — see \$5.7/\$5.9), deliveryMethod (default "AT_EVENT", "SHIPPING" noted as future), stripeCheckoutSessionId? / paystackReference? / paypalOrderId? (each @unique). OrderItem: orderId (FK Cascade), productId? (FK SetNull) XOR ticketTypeId? (FK SetNull) depending on Order.kind, productName/unitPriceCents/quantity (a point-in-time snapshot so later product/ticket-type edits don't rewrite order history).

Article (articles)

authorId (FK → User, no onDelete override specified — INFERRED default Prisma behaviour applies, i.e. restrict; UNKNOWN whether this was tested against actual user deletion), title, slug (@unique, immutable after generation), excerpt, body (plain text with a small markdown-like convention set per code comment, not full rich text), coverImageData?/coverImageMimeType?, status (ArticleStatus: DRAFT, PUBLISHED; default DRAFT), publishedAt? (set/refreshed on each publish — public ordering uses this, not createdAt). Index: [status, publishedAt].

LandingService (landing_services)

title, description, icon (String, constrained to a fixed frontend enum, not free text — see §5.14), sortOrder (default 0), isActive (default true). Index: [isActive, sortOrder].

8.3 Cross-cutting schema conventions

- **IDs:** every model uses cuid() string primary keys, not sequential integers or UUIDs. CONFIRMED, universal across all 27 models.
- **Money:** always stored as an integer count of minor currency units (*Cents fields — costCents, priceCents, totalCents, platformFeeCents, amountCents), never a float, “to avoid rounding drift” per repeated code comments.
- **Currency:** a shared Currency enum (USD | GBP | NGN) attached per-record where money is involved (Vendor, Product, TicketType, EventPayoutAccount, Order, PaymentEvent) — there is no live FX conversion anywhere in the schema or service layer; totals spanning multiple currencies are always reported as an array grouped by currency, never blended.
- **File storage:** every uploadable asset (event cover image, product image, article cover image, invitation card) is stored as a Bytes column directly in Postgres, with a paired *MimeType string column, rather than in an external object store — a deliberate, consistently-applied choice per multiple code comments citing avoidance of separate storage credentials, with file-size caps enforced in the relevant multer upload middleware (5MB for images, 8MB for the invitation card).
- **Public/anonymous trust tokens:** Event.rsvpToken, Event.publicSlug, EventInvitation.token, Ticket.code, and Order.id all function as unguessable bearer credentials granting access without authentication — this is a consistent, repeated pattern rather than several unrelated mechanisms.
- **Soft-hide vs. delete:** Product.active, TicketType.isActive, and LandingService.isActive all provide a way to hide a record from public view without deleting it (preserving order/sales history); Article.status serves the analogous purpose for content.

8.4 Migrations

UNKNOWN — Not determined from the available source code/project artefacts. The apps/api/prisma/migrations directory was not enumerated in this documentation pass; migration history (count, chronology, any manually-edited migration) is therefore not confirmed here. apps/api/package.json confirms the tooling used to manage migrations (prisma migrate dev / prisma migrate deploy npm scripts) and apps/api/railway.json confirms prisma migrate deploy runs automatically on every production boot.

09 — API Documentation

All endpoints below are CONFIRMED directly from the route files listed per module. Base path for every route is /api (mounted in apps/api/src/app.ts). “Auth” column: **Public** = no authentication; **Auth** = requireAuth (valid session, any role); **Admin** = requireAuth + requireAdmin. Response envelope for all endpoints (CONFIRMED, apps/api/src/lib/apiResponse.ts): success responses are {success: true, data: ...}; errors are {success: false, error: {code, message, details?}}.

9.1 Auth — /api/auth (auth.routes.ts)

Method

POST

POST

POST

GET

9.2 Events — /api/events (events.routes.ts)

All routes below router.use(requireAuth); every /:eventId/* route additionally runs auditAdminEventActions().

Method	Path	Purpose
GET	/	List the caller’s events + guest/table summary
POST	/	Create event
GET	/:eventId	Get one event
PUT	/:eventId	Update event (owner or admin)
DELETE	/:eventId	Delete event (owner only, no admin bypass)
GET	/:eventId/dashboard	Per-event stats dashboard
GET	/:eventId/rsvp	RSVP dashboard (delegates to rsvp.controller.dashboard)
GET	/:eventId/invitation-card	Invitation card metadata
GET	/:eventId/invitation-card/file	Download invitation card bytes
POST	/:eventId/invitation-card	Upload/replace invitation card (multipart, ≤8MB)
DELETE	/:eventId/invitation-card	Delete invitation card

GET	/:eventId/cover-image	Download event cover image
POST	/:eventId/cover-image	Upload cover image (multipart, ≤5MB)

Nested under `/:eventId/*` (each its own router, `mergeParams`): guests, seating, vendors, products, orders, payouts, ticket-types — see §9.3–9.9.

9.3 Guests — `/api/events/:eventId/guests` (`guests.routes.ts`) and `/api/guests/:guestId` (`guestById.routes.ts`)

Method

GET
POST
POST
GET
GET
GET
GET
PUT
DELETE
POST
DELETE
POST
GET
POST
POST
POST

`guestById.routes.ts`, mounted at `/api/guests`, exposes GET/PUT/DELETE `/:guestId` again as a top-level, explicitly `requireAuth`-gated alternative path to the same service functions (CONFIRMED — imports the same `guests.controller.ts`).

9.4 Seating — `/api/events/:eventId/seating` (`seating.routes.ts`)

Method	Path	Purpose
GET	/map	Full seating map (layout + tables + unassigned confirmed guests)
GET	/map/export/pdf	Seating map as PDF
GET	/layout	Get (or auto-create) venue layout

PUT	/layout	Update layout settings
POST	/layout/objects	Add a decor object
PUT	/layout/objects/:objectId	Update a decor object
DELETE	/layout/objects/:objectId	Delete a decor object
GET	/tables	List tables
POST	/tables	Create table (auto-creates its seats)
PUT	/tables/:tableId	Update table (may reconcile seat count)
DELETE	/tables/:tableId	Delete table
POST	/assignments	Assign a guest (+ party) to a table/seat
DELETE	/assignments/:guestId	Unassign a guest (+ party)
DELETE	/assignments/ party/:partyMemberId	Unassign a single named party member

9.5 Vendors — /api/events/:eventId/vendors (vendors.routes.ts)

Method

GET
GET
POST
PUT
DELETE

9.6 Merchandise — planner side (/api/events/:eventId/products, /orders) and guest side (/api/shop)

products.routes.ts (planner, auth via parent router):

Method

GET
POST
PUT
DELETE
GET
POST

orders.routes.ts (planner, auth via parent router):

Method

GET
GET
GET

shop.routes.ts (public, mounted at /api/shop):

Method	Path	Rate limit	Purpose
GET	/:token/products	60/min	Public shop listing (by rsvpToken)
GET	/products/:productId/image	60/min	Public product image
POST	/:token/checkout	20/15min	Start checkout
POST	/:token/checkout/paypal/capture	20/15min	Capture an approved PayPal order

webhook.routes.ts (public, mounted directly in app.ts at /api/webhooks, **before** the JSON body parser):

Method

POST
POST

9.7 Payouts — /api/events/:eventId/payouts (payouts.routes.ts)

Method	Path	Purpose
GET	/	List payout accounts (with live Stripe re-sync for incomplete ones)
POST	/stripe/connect	Create/resume Stripe Express onboarding, returns Account Link URL
GET	/paystack/banks	Nigerian bank list (for the connect form dropdown)
POST	/paystack/connect	Connect a Paystack Subaccount
POST	/paypal/connect	Save a PayPal payee email
DELETE	/:payoutAccountId	Disconnect a payout account

9.8 Tickets — planner side (/api/events/:eventId/ticket-types) and public side (/api/tickets)

ticketTypes.routes.ts (planner, auth via parent router):

Method

GET

POST

POST

POST

PUT

DELETE

checkout.routes.ts (public, mounted at /api/tickets):

Method	Path	Rate limit	Purpose
GET	/events/:slug	60/min	Public event + on-sale ticket types
GET	/events/:slug/cover-image	60/min	Public cover image
GET	/orders/:orderId	60/min	Order + issued ticket codes (confirmation page)
POST	/events/:slug/checkout	20/15min	Reserve capacity + start checkout
POST	/events/:slug/checkout/paypal/capture	20/15min	Capture an approved PayPal ticket order

9.9 RSVP — /api/rsvp (rsvp.routes.ts, fully public)

Method	Path	Rate limit	Purpose
GET	/invite/:invitationToken	60/min	Public event + guest prefill via personal invite token
POST	/invite/:invitationToken	20/15min (configurable)	Submit RSVP via personal invite token
GET	/invite/:invitationToken/invitation-card	60/min	Invitation card via invite token
GET	/:token/invitation-card	60/min	Invitation card via shared event token

GET	/:token	60/min	Public event via shared token
POST	/:token	20/15min (configurable)	Submit RSVP via shared token

9.10 Notifications — /api/notifications (notifications.routes.ts, all Auth)

Method

GET
PUT
PUT

9.11 Insights — /api/insights (insights.routes.ts, Auth)

Method	Path	Purpose
GET	/	“Needs Attention” insights, optionally scoped to one eventId query param

9.12 Analytics — /api/analytics (analytics.routes.ts, Auth)

Method

GET

9.13 Admin — /api/admin/* (all Admin-only)

admin.routes.ts (mounted at /api/admin):

Method

GET
GET
GET
GET
GET

articles.admin.routes.ts (mounted at /api/admin/articles):

Method

GET
POST
GET
PUT
DELETE
POST

POST
GET
POST

landing.admin.routes.ts (mounted at /api/admin/services):

Method

GET
POST
POST
PUT
DELETE

9.14 Public content — /api/articles, /api/landing

articles.public.routes.ts (mounted at /api/articles, Public):

Method

GET
GET
GET

landing.public.routes.ts (mounted at /api/landing, Public):

Method

GET

9.15 Platform

Method	Path	Auth	Purpose
GET	/health	Public	Liveness + DB connectivity check (mounted directly in app.ts, not under /api)

9.16 Endpoint count

Counting every row in the tables above (including both guests.routes.ts and guestById.routes.ts guest-by-id paths as distinct mounts, and the /health liveness route): **122 endpoints** across **22 route files** (21 module route files plus webhook.routes.ts, mounted directly in app.ts). See 22-application-inventory.md for the full counting methodology and per-module breakdown.

10 — Roles and Permissions

10.1 Roles

The schema defines exactly two roles via UserRole (CONFIRMED, apps/api/prisma/schema.prisma): PLANNER (default for every new registration) and ADMIN. There is no self-service path to become ADMIN — no route accepts a client-supplied role value on registration (auth.schema.ts registerSchema has no role field). How the first admin account is created is UNKNOWN from the routes/services reviewed in this pass — Not determined from the available source code/project artefacts (likely a direct database update or a seed script; apps/api/prisma/seed.ts is referenced in package.json’s seed script but its contents were not read in this documentation pass).

A third “role” — **unauthenticated guest/ticket buyer** — is not a database role at all; it is anyone reaching a public route (/api/rsvp/*, /api/shop/*, /api/tickets/*, /api/articles/*, /api/landing/*) with no session, authorized purely by possessing an unguessable token/slug (see 14-security-documentation.md §14.3).

10.2 Permission matrix

Capability	PLANNER (own event)	PLANNER (other’s event)	ADMIN (own event)	ADMIN (other’s event)	Unauthentic ated
Register / log in					(register/log in itself)
View/edit own profile (/auth/me)		—		—	
Create event		—		—	
Read event		(404)			
Update event		(404)			
Delete event		(404)		(404 — no admin bypass)	
Guests / seating / vendors / merchandise / ticket types CRUD		(404)		(audited)	
Connect/		(404)		(404 — no	

change payout account (Stripe/Paystack/PayPal)					admin bypass)	
View own orders/payment events for an event		(404)				
Submit an RSVP (via token)	—	—	—	—	—	(rate-limited)
Browse merchandise shop / buy (via token)	—	—	—	—	—	(rate-limited)
Buy public tickets (via slug)	—	—	—	—	—	(rate-limited)
Door check-in scan (guest QR or ticket code)	(own event)				(audited)	
List all subscribers / all events (/admin/users, /admin/events)						
View admin audit log / cross-subscriber payment events						
View platform-wide analytics						
Author/publish/delete						

articles
Manage
landing
“Services”
cards
Read
published
articles /
active
services /
landing
content

CONFIRMED — derived directly from `apps/api/src/middleware/auth.ts` (`requireAuth`, `requireAdmin`), `apps/api/src/modules/events/events.service.ts` (`getOwnedEvent` vs `getOwnedEventStrict`), `apps/api/src/modules/payouts/payouts.service.ts` (all three `connect*` functions call `getOwnedEventStrict`), and the route-level `requireAdmin` gating on `admin.routes.ts`, `articles.admin.routes.ts`, `landing.admin.routes.ts`.

10.3 Notes on the “admin bypass” model

Rather than a parallel `/api/admin/events/:id/*` API, EventFlow’s admin support access works by having the **owner-scoped** endpoints (`/api/events/:eventId/*` and everything nested under it) accept an ADMIN caller as if they owned the event, via `getOwnedEvent`’s explicit role check. This is a deliberate architectural choice (INFERRED intent from code comments in `events.service.ts`: “so an ADMIN passed in here gets read/write access everywhere those modules already allow the owner to act, by design”) that guarantees admin support access can never functionally diverge from what a planner can do to their own event — there is no separate admin code path to fall out of sync. Two specific actions are explicitly excluded from this bypass (owner-only, enforced via `getOwnedEventStrict`): deleting an event outright, and creating/changing a payout account’s financial connection details.

10.4 Audit trail scope

`auditAdminEventActions` (CONFIRMED, `apps/api/src/middleware/auditAdminEventActions.ts`) logs **only** when all of the following are true: the request method is not GET, the response status is `< 400`, the requester’s role is ADMIN, and the target event’s `userId` is **not** the requester’s own id. A planner acting on their own event is never logged by this middleware (there is nothing anomalous to record). An admin’s read-only (GET) actions on a subscriber’s event are also never logged — only mutations are audited.

11 — Business Rules

Rules extracted from service-layer logic. All CONFIRMED against the cited file unless marked otherwise.

Access & ownership

- **BR-001:** A user can only read or mutate an event if they own it, or if they hold the ADMIN role. All other requesters receive a 404 (not 403), so event existence is never disclosed to an unauthorized caller. `events.service.ts getOwnedEvent`.
- **BR-002:** Deleting an event, and connecting/modifying a payout account, are permitted for the owner only — the ADMIN bypass does not apply to these two actions. `events.service.ts getOwnedEventStrict`; `payouts.service.ts`.
- **BR-003:** A ticket type with `quantitySold > 0` cannot be deleted; it must be set inactive instead. `ticketTypes.service.ts deleteTicketType`.

RSVP & guests

- **BR-010:** An RSVP submission is rejected if the event's `rsvpOpen` flag is false, or if `rsvpDeadline` has passed. `rsvp.service.ts checkRsvpIsOpen`.
- **BR-011:** When matching a shared-link RSVP submission to an existing invited guest, the system matches by email first (case-insensitive exact match), then by first+last name (case-insensitive exact match), and only creates a new guest if neither matches. `rsvp.service.ts submitRsvp`.
- **BR-012:** A guest's `additionalGuestsCount` is only meaningful when `rsvpStatus = CONFIRMED`; a non-confirmed submission always resets it to 0. `rsvp.service.ts guestUpdateData`.
- **BR-013:** Whenever a guest's RSVP status transitions away from CONFIRMED, any existing seat assignment for that guest and for their whole named party is deleted. `utils/rsvpMath.ts shouldReleaseSeatOnStatusChange`; enforced in both `guests.service.ts updateGuest` and `rsvp.service.ts submitRsvp/submitRsvpViaInvitation`.
- **BR-014:** Updating a guest's `additionalGuestNames` array fully replaces the existing named-party list (delete-all, then re-create) rather than diffing individual entries.

Seating & capacity

- **BR-020:** A guest cannot be assigned to a table if doing so (accounting for their whole party's size) would exceed the table's capacity, unless the caller explicitly sets `overrideCapacity = true`. `utils/capacity.ts canAssignGuest`.
- **BR-021:** A guest who declined the invitation may still be assigned a seat, with a non-blocking warning returned to the caller — this is allowed by design, not an oversight. `utils/capacity.ts canAssignGuest`.
- **BR-022:** A guest can hold at most one active seat assignment at a time; assigning them elsewhere clears the prior assignment as part of the same operation.

seating.service.ts assignGuest (relies on the @unique constraint on SeatingAssignment.guestId).

- **BR-023:** Reducing a table's capacity below its currently occupied seat count unseats occupants starting from the highest seat number; if an unseated seat belonged to a guest or a named party member, that guest's **entire party** is unassigned as a unit, never partially. seating.service.ts updateTable.
- **BR-024:** Named party members are seated as close as possible to their primary guest's seat (searching outward, alternating +1/-1 seat-number offset, wrapping around the table), not assigned arbitrarily. seating.service.ts assignGuest.

Vendors & merchandise

- **BR-030:** Vendor and product costs are tracked per-currency; totals spanning multiple currencies within one event are always reported as a per-currency breakdown, never summed into one blended figure. vendors.service.ts groupCostsByCurrency; analytics.service.ts (identical local helper).
- **BR-031:** A cart cannot mix items priced in different currencies; checkout is rejected outright if it does. orders.service.ts createCheckoutSession; tickets/checkout.service.ts createTicketCheckoutSession.
- **BR-032:** Product stock is decremented only when an order reaches PAID status (confirmed by the payment processor), never at checkout initiation — an abandoned checkout never reduces available stock. orders.service.ts finalizeOrderPaid.
- **BR-033:** A product hidden via active = false remains visible in the planner's own product list and order history, but disappears from the public shop listing.

Payments & payouts

- **BR-040:** Checkout may only proceed against a currency for which the event has at least one EventPayoutAccount that is "connected" — Stripe Connect requires completed onboarding, Paystack requires a saved subaccount code, PayPal requires a saved payee email. payouts.service.ts isPayoutAccountConnected.
- **BR-041:** If the buyer does not explicitly choose a payment provider, the system defaults to the first available in the fixed order Stripe Connect — Paystack — PayPal. orders.service.ts DEFAULT_PROVIDER_PREFERENCE.
- **BR-042:** The platform fee percentage applied to an order is read from configuration at the moment of checkout and frozen onto that order (Order.platformFeeCents); a later change to the configured percentage never retroactively changes an already-placed order's fee. apps/api/prisma/schema.prisma (Order.platformFeeCents comment); orders.service.ts.
- **BR-043:** Merchandise orders and ticket orders use independently configurable platform fee percentages (PLATFORM_FEE_PERCENT vs. TICKET_FEE_PERCENT), both currently defaulted to 5%. apps/api/src/config/env.ts.
- **BR-044:** If PayPal rejects an attempt to attach a platform fee (no Partner/BN-code enrollment), the system retries the same order without a fee rather than failing the checkout, and records platformFeeCents = 0 on that order to match what was

actually collected. lib/paypalClient.ts createPaypalOrder; orders.service.ts startPaypalCheckout.

- **BR-045:** An order is only ever marked PAID as a result of a signature-verified webhook (Stripe/Paystack) or a server-initiated PayPal capture call — never from a client-supplied “I paid” signal. orders.service.ts handleStripeWebhook, handlePaystackWebhook, capturePaypalOrderCore.
- **BR-046:** Every payment provider event received — success, decline, failure, or expiry — is logged to PaymentEvent, independent of whether it results in a change to order status. orders.service.ts logPaymentEvent.
- **BR-047:** A Paystack subaccount’s platform fee percentage (percentage_charge) is fixed at the moment the planner connects it; changing the global fee configuration afterward does not alter already-connected subaccounts. payouts.service.ts connectPaystack.
- **BR-048:** Raw bank account numbers are never persisted in the application database, for any provider. apps/api/prisma/schema.prisma (EventPayoutAccount comment); payouts.service.ts connectPaystack (account number sent to Paystack once, not stored).

Ticketing

- **BR-050:** Ticket-type capacity is reserved atomically at checkout-session creation (not at payment confirmation) via a single conditional SQL UPDATE, making concurrent checkouts for the same limited-capacity ticket type race-safe. ticketTypes.service.ts reserveTickets.
- **BR-051:** If a checkout attempt fails after capacity has been reserved (processor rejection) or a reserved order later expires/is declined, the reserved capacity is released back to the ticket type. orders.service.ts releasePendingTicketOrder; tickets/checkout.service.ts createTicketCheckoutSession catch block.
- **BR-052:** A ticket purchase must respect the ticket type’s minPerOrder/maxPerOrder bounds and its sales window (salesStartAt/salesEndAt), re-validated server-side at checkout regardless of what the public listing displayed. tickets/checkout.service.ts createTicketCheckoutSession.
- **BR-053:** One Ticket row (with its own unique, unguessable code) is issued per unit purchased, only once the order is confirmed PAID. orders.service.ts finalizeOrderPaid.
- **BR-054:** Scanning an already-CHECKED_IN ticket does not error, but is flagged (alreadyCheckedIn: true) to the scanning staff as a possible duplicate/shared ticket; scanning a CANCELLED ticket is rejected outright. ticketTypes.service.ts checkInTicketByCode.

Content & publishing

- **BR-060:** An event’s publicSlug and an article’s slug are each generated once (from the event/article name/title, slugified, with a numeric collision suffix if needed) and are immutable thereafter, so previously shared public links never break due to a later

rename. events.service.ts ensureUniqueEventSlug; articles.service.ts ensureUniqueSlug.

- **BR-061:** Published articles are ordered publicly by publishedAt, not createdAt — republishing an older draft moves it back to the top of the public list. articles.service.ts listPublishedArticles.
- **BR-062:** A LandingService card's icon value must be one of a fixed, frontend-enumerated set — free-text/SVG icon input is not accepted, to avoid it becoming an injection vector. apps/web/src/types/index.ts (SERVICE_ICON_OPTIONS); code comment in landing.schema.ts.

12 — Testing Documentation

12.1 Test strategy (as implemented)

CONFIRMED — apps/api/package.json scripts (test, test:watch, test:integration), apps/api/vitest.config.ts / apps/api/vitest.integration.config.ts (existence confirmed; contents not read in this pass), .github/workflows/ci.yml.

The backend (apps/api) has two distinct automated test tiers, both using Vitest:

1. **Unit tests** (apps/api/tests/unit/*.test.ts) — test pure business-logic functions with no database, run via npm test / vitest.config.ts.
2. **Integration tests** (apps/api/tests/integration/*.test.ts) — exercise the Express app end-to-end via Supertest against a real PostgreSQL database, run via vitest run --config vitest.integration.config.ts. apps/api/tests/setup/globalSetup.ts exists (CONFIRMED by file listing) to provision the database for this tier; embedded-postgres is listed as a devDependency in apps/api/package.json, consistent with spinning up an ephemeral local Postgres for integration runs, though the exact mechanism (embedded-postgres locally vs. the CI Postgres service container) was not verified by reading globalSetup.ts line-by-line in this pass — INFERRED from dependency presence and CI configuration, not directly confirmed.

The frontend (apps/web) has **no automated test files** — CONFIRMED by the absence of any test/vitest/jest script in apps/web/package.json (only dev, build, lint, typecheck, preview) and no test runner listed among its dependencies. Frontend quality gating in CI is limited to typecheck, lint, and build succeeding (see 13-test-automation-documentation.md).

There is no end-to-end (browser-driven, e.g. Playwright/Cypress) test suite anywhere in the repository — UNKNOWN/not present, confirmed by the absence of any such dependency in either package.json.

12.2 Existing automated test inventory

File	Type	Module(s) covered
tests/unit/rsvpMath.test.ts	Unit	utils/rsvpMath.ts — RSVP stat computation, seat-release rule
tests/unit/capacity.test.ts	Unit	utils/capacity.ts — table capacity/assignment decision rules
tests/unit/inviteEmail.test.ts	Unit	Invite email HTML/content generation logic
tests/integration/auth.test.ts	Integration	Register/login/logout/me

tests/integration/ events.test.ts	Integration	Event CRUD, ownership
tests/integration/ guests.test.ts	Integration	Guest CRUD, list/filter, CSV import/export, check-in
tests/integration/ rsvp.test.ts	Integration	Public RSVP submission flow
tests/integration/ seating.test.ts	Integration	Layout, tables, seat assignment/unassignment
tests/integration/ invitationCard.test.ts	Integration	Invitation card upload/download/delete
tests/integration/ vendors.test.ts	Integration	Vendor CRUD, summary
tests/integration/ notifications.test.ts	Integration	Notification listing, mark read
tests/integration/ insights.test.ts	Integration	“Needs Attention” insight generation
tests/integration/ analytics.test.ts	Integration	Cross-event analytics overview
tests/integration/ merchandise.test.ts	Integration	Products, checkout, orders (merchandise)
tests/integration/ payouts.test.ts	Integration	Payout account connect/list/disconnect
tests/integration/ ticketTypes.test.ts	Integration	Ticket type CRUD (and, by route co-location, likely the /scan door check-in endpoint — CONFIRMED file exists; exact scenario coverage inside it was not read line-by-line this pass)
tests/integration/ admin.test.ts	Integration	Admin user/event listing, audit log, payment events, platform analytics
tests/integration/ articles.test.ts	Integration	Article admin CRUD + public read
tests/integration/ landingServices.test.ts	Integration	Landing service admin CRUD + public read
tests/helpers/ authHelpers.ts	Helper	Shared login/token helpers for integration tests
tests/helpers/testApp.ts	Helper	Shared Express app instance for Supertest
tests/setup/ globalSetup.ts	Setup	Test database provisioning

CONFIRMED — full file listing via Glob `apps/api/tests/**/*.ts`, 21 files total (3 unit + 16 integration + 2 helpers + 1 setup — note this differs from the file *count* cited elsewhere for “distinct test files with assertions,” which is 19: 3 unit + 16 integration).

12.3 Confirmed coverage gap: public ticket checkout

No test file corresponds to `apps/api/src/modules/tickets/checkout.service.ts` or `checkout.routes.ts` (public ticket purchase: capacity reservation under concurrency, provider routing, PayPal capture, order finalization → ticket issuance). This is a **confirmed gap**, not a speculative one — the file listing in §12.2 is exhaustive for `apps/api/tests/**/*.ts`, and no file named anything resembling `ticketCheckout/checkout/ticketOrders` exists. This matches this project’s own internal task tracking, which records “Backend tests: ticket checkout + issuance + door scan” as still pending. See 20-known-issues-risks-technical-debt.md for risk framing.

12.4 Coverage matrix (module vs. test tier)

Module	Unit test	Integration test	Notes
Auth	—	<code>auth.test.ts</code>	
Events	—	<code>events.test.ts</code>	
Guests	—	<code>guests.test.ts</code>	
RSVP	<code>rsvpMath.test.ts</code> (pure logic)	<code>rsvp.test.ts</code>	
Invitations / email	<code>inviteEmail.test.ts</code>	—	No integration test found for <code>invite.service.ts</code> ’s <code>getInviteLink/sendInviteEmail/bulkSendInviteEmails</code> HTTP endpoints specifically — INFERRED gap, not exhaustively cross-checked against <code>guests.test.ts</code> contents in this pass
Invitation card	—	<code>invitationCard.test.ts</code>	
Seating	<code>capacity.test.ts</code> (pure logic)	<code>seating.test.ts</code>	
Vendors	—	<code>vendors.test.ts</code>	
Merchandise	—		Webhook signature-

(products/orders/webhooks)	—	merchandise.test.ts	failure paths specifically not confirmed covered — see §12.5
Payouts	—	payouts.test.ts	
Ticket types (CRUD + scan)	—	ticketTypes.test.ts	
Ticket checkout (public purchase flow)	—	None found	Confirmed gap, §12.3
Notifications	—	notifications.test.ts	
Insights	—	insights.test.ts	
Analytics (planner)	—	analytics.test.ts	
Admin (users/events/audit/payment-events/platform analytics)	—	admin.test.ts	
Articles	—	articles.test.ts	
Landing services	—	landingServices.test.ts	
Frontend (all of apps/web)	—	—	No automated frontend tests exist at all

12.5 Scenario catalogue: existing vs. recommended-but-missing

The following distinguishes scenarios this documentation effort has direct evidence are tested (by file existence — exact assertions inside each file were not individually re-verified line-by-line in this pass beyond what's cited elsewhere in this pack) from scenarios that appear untested based on the absence of any corresponding file/section.

Existing (file-confirmed): - Registration/login happy path and duplicate-email/invalid-credential rejection (auth.test.ts). - Event ownership enforcement (events.test.ts). - RSVP submission via shared token, deadline enforcement (rsvp.test.ts, rsvpMath.test.ts). - Seat assignment capacity rules, party seating (seating.test.ts, capacity.test.ts). - Vendor CRUD and currency-grouped summary (vendors.test.ts). - Merchandise checkout and webhook-driven order finalization (merchandise.test.ts). - Payout account connection flows (payouts.test.ts). - Ticket type CRUD (ticketTypes.test.ts). - Admin cross-subscriber listing, audit log, platform analytics (admin.test.ts).

Recommended but missing (no corresponding test file/section found): - Public ticket checkout: concurrent reservation race (two buyers, last ticket), provider routing, PayPal capture, finalizeOrderPaid issuing correct ticket count. See §12.3. - Door check-in scan for both guests (checkInGuestByToken) and tickets (checkInTicketByCode), specifically the idempotent re-scan (alreadyCheckedIn) and cancelled-ticket-rejection branches — INFERRED as likely partially covered inside guests.test.ts/ticketTypes.test.ts given the endpoints live in those route files, but not confirmed by direct reading of assertions in this pass. - Stripe/Paystack webhook **signature verification failure** paths (wrong/missing signature → 400) — not confirmed present or absent without reading merchandise.test.ts's assertions directly; flagged as recommended regardless, since this is a security-relevant boundary. - Any frontend behaviour whatsoever (form validation, seating canvas drag interactions, checkout UI state) — no frontend test tooling exists to test it with. - Rate-limiting behaviour (that limits actually trigger a 429/RATE_LIMITED response under load) — the rate limiters are configured (CONFIRMED, route files) but no test file was found asserting their triggering behaviour. - CSV import with malformed/partial rows (see 24-documentation-gap-analysis.md for why this specific behaviour is UNKNOWN).

12.6 CI enforcement of tests

Both the unit and integration suites run on every push/PR to main/develop via the api job in .github/workflows/ci.yml, against a real postgres:16-alpine service container (not a mock), after prisma generate and prisma migrate deploy. A failing test fails the CI job, which (INFERRED standard GitHub branch-protection behaviour, not itself confirmed configured) would typically block a PR merge if branch protection rules require the check — whether such branch protection rules are actually enabled on this repository is UNKNOWN from the files reviewed (branch protection is a GitHub repository setting, not something expressed in the .yml file itself).

13 — Test Automation Documentation

13.1 Frameworks and tooling

Tool	Version	Purpose	Evidence
Vitest	^2.1.1	Test runner (unit + integration)	apps/api/package.json
Supertest	^7.0.0	HTTP-level assertions against the Express app in integration tests	apps/api/package.json
embedded-postgres	^18.4.0-beta.17	Devdependency, consistent with an ephemeral local Postgres for integration test runs	apps/api/package.json
TypeScript / tsx	^5.6.2 / ^4.19.1	Test files are TypeScript, executed via Vitest's native TS support	apps/api/package.json

No test framework (Vitest, Jest, or otherwise) is present in apps/web/package.json — the frontend has zero automated test infrastructure. CONFIRMED by absence.

13.2 Configuration files

- apps/api/vitest.config.ts — unit test configuration. Existence CONFIRMED (npm test script points here); contents not read in this documentation pass.
- apps/api/vitest.integration.config.ts — integration test configuration. Existence CONFIRMED (npm run test:integration script points here); contents not read in this pass.
- apps/api/tests/setup/globalSetup.ts — referenced by prior investigation in this project as responsible for provisioning a test database and writing a .env.test file before any test file is imported (CONFIRMED indirectly: apps/api/src/config/env.ts explicitly branches on NODE_ENV === "test" to load .env.test). Full contents not re-read line-by-line in this pass — treat the exact provisioning mechanism (embedded-postgres binary vs. assuming an already-running Postgres) as INFERRED, not CONFIRMED.

13.3 How to run the tests locally

Derived directly from apps/api/package.json scripts and root package.json workspace scripts (CONFIRMED):

```
# From repo root, install all workspace dependencies
npm install
```

Unit tests only

`npm run test --workspace=apps/api`

or: cd apps/api && npm test

Integration tests (requires a reachable Postgres — see globalSetup.ts /

docker-compose.yml for local Postgres)

`cd apps/api && npm run test:integration`

Typecheck (both apps)

`npm run typecheck:api`

`npm run typecheck:web`

Lint (both apps)

`npm run lint:api`

`npm run lint:web`

The frontend has no test script — `npm run lint` / `npm run typecheck` / `npm run build` are the only automated quality gates available for `apps/web`.

13.4 CI pipeline wiring

CONFIRMED — `.github/workflows/ci.yml`. Two independent jobs, both triggered on push/PR to main or develop:

api job (ubuntu-latest, with a postgres:16-alpine service container): 1. Checkout, Node 20 setup with npm cache. 2. `npm install` (root, installs both workspaces). 3. `npx prisma generate` (in `apps/api`). 4. `npx prisma migrate deploy` (in `apps/api`) — applies real migrations against the CI Postgres container, not a mock. 5. `npm run typecheck` (`apps/api`). 6. `npm run lint` (`apps/api`). 7. `npm test` (unit tests). 8. `npx vitest run --config vitest.integration.config.ts` (integration tests). 9. `npm run build` (`apps/api`).

Environment for this job is hardcoded in the workflow file: `DATABASE_URL` pointing at the local service container, `NODE_ENV=test`, `JWT_SECRET=ci-test-secret`, `JWT_EXPIRES_IN=1h`, `COOKIE_NAME=event_rsvp_token`, `CORS_ORIGINS=http://localhost:5173`, `PUBLIC_APP_URL=http://localhost:5173`, `RSVP_RATE_LIMIT_MAX=1000`, `RSVP_RATE_LIMIT_WINDOW_MS=900000` (rate limit deliberately loosened for CI so test runs aren't throttled).

web job (ubuntu-latest): 1. Checkout, Node 20 setup. 2. `npm install`. 3. `npm run typecheck` (`apps/web`). 4. `npm run lint` (`apps/web`). 5. `npm run build` (`apps/web`), with `VITE_API_URL=https://placeholder-api.example.com/api` — a placeholder value, since the build doesn't need a real API to succeed (Vite build is static).

Neither job publishes coverage reports, uploads build artifacts, or deploys anything — CI here is verification-only; deployment is a separate, platform-triggered process (see 15-deployment-and-infrastructure.md).

13.5 What is not automated

- No coverage threshold enforcement (no `--coverage` flag or coverage tool configured in either `vitest.config.ts` reference in `package.json` scripts).
- No mutation testing, contract testing, load testing, or accessibility testing (axe, Lighthouse CI, etc.) tooling present.
- No pre-commit hook framework (e.g. Husky) found in either `package.json`'s dependencies — CONFIRMED by absence in both files reviewed.
- No automated dependency-vulnerability scanning step in `ci.yml` (e.g. `npm audit`, Dependabot/Snyk workflow) — UNKNOWN whether Dependabot or similar is configured at the GitHub repository-settings level (not expressible in a workflow YAML file, so not verifiable from source alone).

14 — Security Documentation

No penetration testing, fuzzing, dependency scanning, or destructive testing was performed to produce this document. Everything below is static code review only, evidence-cited. This document does not certify the application secure; it records what controls exist in the code and flags where evidence is insufficient to judge.

14.1 Implemented controls (CONFIRMED)

Authentication

- Passwords hashed with bcrypt at 12 salt rounds before storage; never stored or logged in plaintext. `apps/api/src/utils/password.ts`.
- Sessions are JWTs signed with `JWT_SECRET` (HMAC, via `jsonwebtoken`), default 7-day expiry, delivered as an `httpOnly` cookie (`secure: true` and `sameSite: "none"` in production, mitigating some XSS-driven token theft and enabling cross-site cookie delivery for the separately-hosted Vercel frontend) with a Bearer header fallback for contexts where the cookie isn't usable.
`apps/api/src/modules/auth/auth.controller.ts`, `apps/api/src/middleware/auth.ts`.
- Login failure messages are identical for “no such user” and “wrong password,” reducing user-enumeration risk via the login endpoint.
`apps/api/src/modules/auth/auth.service.ts`.
- Registration/login are rate-limited (30 requests / 15 minutes).
`apps/api/src/modules/auth/auth.routes.ts`.

Authorization

- Every event-scoped mutation passes through a single ownership choke point (`getOwnedEvent/getOwnedEventStrict`), rather than each module re-implementing its own ownership check.
`apps/api/src/modules/events/events.service.ts`.
- Admin-only routes are gated by `requireAdmin`, which performs a fresh database role lookup per request (not a role claim trusted from the JWT payload itself — the JWT only carries `userId`). `apps/api/src/middleware/auth.ts`.
- Two specific admin-support-relevant actions (delete event, connect/modify payout account) are excluded from the admin ownership bypass at the service layer, not just hidden in the UI. See 11-business-rules.md BR-002.

Transport / headers

- `helmet()` applied globally, providing standard security headers (CSP defaults, X-Content-Type-Options, etc. — exact Helmet default policy not individually enumerated in this pass; INFERRED from Helmet's documented defaults for the version installed, not re-derived from Helmet's own source). `apps/api/src/app.ts`.

- CORS restricted to an explicit, configured origin allow-list (CORS_ORIGINS env var) with credentials: true (required for the cookie-based session to work cross-origin between Vercel and Railway). apps/api/src/app.ts, apps/api/src/config/env.ts.
- app.set("trust proxy", 1) is set, needed for correct client IP resolution (used by rate limiting) behind Railway's reverse proxy. apps/api/src/app.ts.

Input validation

- Every request body/query/params that a route accepts is validated (and its type coerced) via a Zod schema through validateBody/validateQuery/validateParams middleware before reaching a controller. apps/api/src/middleware/validate.ts, used consistently across all 22 route files reviewed.
- Zod validation failures return a structured 400 VALIDATION_ERROR with field-level detail (err.flatten()), not a raw stack trace. apps/api/src/middleware/errorHandler.ts.

Payment webhook integrity

- Stripe webhook payloads are verified via stripe.webhooks.constructEvent against STRIPE_WEBHOOK_SECRET (and optionally a second STRIPE_CONNECT_WEBHOOK_SECRET), using the untouched raw request body (mounted before the JSON parser specifically for this reason). apps/api/src/modules/products/orders.service.ts, apps/api/src/modules/products/webhook.routes.ts.
- Paystack webhook payloads are verified via an HMAC-SHA512 signature comparison against PAYSTACK_SECRET_KEY, also using the raw body. apps/api/src/modules/products/orders.service.ts (handlePaystackWebhook).
- No order is ever transitioned to PAID by a client-supplied request — only by a verified webhook or a server-initiated PayPal capture call. See 11-business-rules.md BR-045.

Data minimization / secrets handling

- Raw bank account numbers are never persisted to the application database for any payment provider (see BR-048).
- Uploaded files are type- and size-restricted at the multer middleware layer per upload endpoint (5MB for images, 8MB for invitation cards) before ever reaching a service function.
- Error responses hide internal error messages in production (env.isProduction check both in the global error handler and the /health endpoint), surfacing only a generic message; full messages are logged server-side via pino, not returned to the client. apps/api/src/middleware/errorHandler.ts, apps/api/src/app.ts.

Public/anonymous access model

- Every anonymous-access surface (RSVP, shop, ticket purchase, invitation card download) is gated by an unguessable, randomly-generated token or slug (cuid()-derived), not by a predictable or sequential identifier. See 08-database-documentation.md §8.3.

- Public read endpoints (articles, landing services, RSVP reads, shop reads, ticket reads) are individually rate-limited (60–120 requests/minute depending on endpoint) to reduce scraping/enumeration throughput. See 09-api-documentation.md for exact limits per route.

Rate limiting summary

Surface	Limit	Evidence
Auth register/login	30 / 15 min	auth.routes.ts
RSVP submit	RSVP_RATE_LIMIT_MAX (default 20) / RSVP_RATE_LIMIT_WIN DOW_MS (default 15 min)	rsvp.routes.ts
RSVP reads	60 / min	rsvp.routes.ts
Shop reads	60 / min	shop.routes.ts
Shop checkout	20 / 15 min	shop.routes.ts
Ticket reads	60 / min	checkout.routes.ts (tickets)
Ticket checkout	20 / 15 min	checkout.routes.ts (tickets)
Article reads	120 / min	articles.public.routes.ts
Landing reads	60 / min	landing.public.routes.ts

14.2 Potential gaps (observed, not confirmed exploitable — no testing performed)

- **Logout does not invalidate the JWT.** POST /api/auth/logout clears the cookie client-side but the API has no token blacklist/revocation mechanism — a token or cookie value captured before logout (e.g. via a proxy, browser extension, or XSS prior to any CSP mitigation taking effect) remains valid for up to 7 days after the user “logs out.” This is a design characteristic of stateless JWTs without a revocation list, not a bug per se, but is worth flagging given the 7-day default expiry. apps/api/src/modules/auth/auth.controller.ts, apps/api/src/utils/jwt.ts.
- **No visible CSRF-specific mitigation for cookie-authenticated state-changing requests.** The cookie is sameSite: “none” in production (required for the cross-origin Vercel→Railway setup), which does not itself prevent CSRF; the code relies on the CORS origin allow-list plus the Bearer-token fallback pattern, but no explicit CSRF token/double-submit-cookie mechanism was found in apps/api/src/middleware/*. Whether the CORS allow-list alone is sufficient mitigation for this app’s threat model is a judgment call outside the scope of a static documentation pass — flagged, not resolved, here.
- **AdminAuditLog writes are best-effort and fire-and-forget** (auditAdminEventActions.ts): a failure to write the audit row is silently swallowed and never surfaced, retried, or alerted on. An admin action could succeed while its

audit trail entry silently fails to be written, with no operator-visible signal that this happened.

- **CSV import validation depth is UNKNOWN** (see 24-documentation-gap-analysis.md) — the controller-level parsing/validation of uploaded CSV rows was not read line-by-line in this pass, so whether malformed or maliciously crafted CSV content (e.g. formula-injection payloads consumed later by a spreadsheet export) is sanitized cannot be confirmed either way.
- **No explicit password-reset / account-recovery flow was found** among the 4 auth routes (register, login, logout, me) — a user who forgets their password has no self-service recovery path in the reviewed code. This may be a genuine functional gap rather than a security control gap per se; cross-referenced in 20-known-issues-risks-technical-debt.md.
- **No account lockout after repeated failed logins** beyond the blanket 30-requests/15-minutes rate limit on the whole /register+/login pair — there is no per-account lockout counter, only a per-client-IP request-rate limit (which is itself trivially distributed across many IPs).

14.3 Unable to determine from source (UNKNOWN)

- Actual production values of any secret (JWT_SECRET, STRIPE_SECRET_KEY, DATABASE_URL, etc.) — appropriately, these are not present in the repository at all; see 16-configuration-and-environment.md for the full redacted list.
- Whether GitHub branch protection, required reviews, or secret-scanning are enabled on the repository — these are platform/organization settings, not expressible in the files reviewed.
- Whether the production Railway/Vercel environments have any additional network-level protections (WAF, IP allow-listing, DDoS mitigation) beyond what Railway/Vercel provide by default — Not determined from the available source code/project artefacts.
- Real-world incident history, whether any of the gaps above have ever been exploited, or the current patch/CVE status of third-party dependencies at the time of reading — Not determined from the available source code/project artefacts; see 17-dependencies-and-technology-stack.md for the dependency list as a starting point for such an assessment, which this pack does not itself perform.
- Session fixation, timing-attack resistance of the bcrypt/JWT comparison paths, and other properties that require dynamic/runtime testing rather than static reading — out of scope for this document by design (no testing was performed).

15 — Deployment & Infrastructure

15.1 Hosting summary

Component	Platform	Evidence
API (apps/api)	Railway, Docker build	apps/api/railway.json
Frontend (apps/web)	Vercel, static build	apps/web/vercel.json
Database	PostgreSQL (Railway-managed, INFERRED from co-location with the API service; not independently confirmed as a distinct managed offering vs. a Railway Postgres plugin)	apps/api/prisma/schema.prisma (provider = "postgresql")
CI	GitHub Actions	.github/workflows/ci.yml
Local dev database	Docker Compose, postgres:16-alpine	docker-compose.yml

15.2 API deployment (Railway)

CONFIRMED — apps/api/railway.json:

```
{
  "build": { "builder": "DOCKERFILE", "dockerfilePath": "Dockerfile" },
  "deploy": {
    "startCommand": "sh -c \"... npx prisma migrate deploy; ... exec node dist/index.js\"",
    "healthcheckPath": "/health",
    "healthcheckTimeout": 30,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3
  }
}
```

- Build uses a Dockerfile (apps/api/Dockerfile — existence CONFIRMED by this configuration referencing it; contents not read in this pass).
- On every boot, the start command runs prisma migrate deploy with a 25-second timeout **before** starting the server; the migration exit code is logged but does not block the server from starting even if migrations fail or time out (exec node dist/index.js runs unconditionally after, per the shell command's structure — CONFIRMED by reading the literal startCommand string). This means a failed migration on deploy will not necessarily surface as a failed deployment; it would need to be caught by watching deploy logs or by the application later failing at runtime due to a schema mismatch.

- Railway health-checks GET /health (30-second timeout) and restarts the container up to 3 times on failure (ON_FAILURE restart policy).
- /health itself runs SELECT 1 against the database and returns 503 with database: "disconnected" if that fails, 200 with database: "connected" otherwise.
apps/api/src/app.ts.

15.3 Frontend deployment (Vercel)

CONFIRMED — apps/web/vercel.json:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

- npm run build runs tsc -b && vite build (apps/web/package.json), i.e. a project-references TypeScript build followed by the Vite production bundle.
- Output directory dist is served as static assets.
- A catch-all rewrite sends every path to index.html, which is required for client-side routing (React Router) to work correctly on hard-refresh/direct navigation to a nested route like /events/abc123/guests.
- The VITE_API_URL environment variable (build-time, embedded into the static bundle by Vite) must point at the deployed API's public URL in the real Vercel project configuration; in CI it is set to a harmless placeholder purely so the build step succeeds (see 13-test-automation-documentation.md §13.4).

15.4 Local development

CONFIRMED — docker-compose.yml, root package.json:

```
docker compose up -d      # starts postgres:16-alpine on :5432
npm install               # root, installs both workspaces
npm run dev:api           # apps/api, tsx watch
npm run dev:web           # apps/web, vite dev server
```

docker-compose.yml defines exactly one service (postgres) with a named volume (event-rsvp-pgdata) for persistence and a healthcheck (pg_isready). No other local services (Redis, message queue, mail catcher, storage emulator) are defined — consistent with the “no external dependencies beyond Postgres and third-party payment/email APIs” architecture observed throughout the codebase.

15.5 CI/CD relationship

GitHub Actions (.github/workflows/ci.yml) runs verification only (typecheck/lint/test/build for both apps) on push/PR to main/develop — it does **not** perform the actual deployment. Deployment to Railway and Vercel is therefore driven by each platform's own native Git-integration (INFERRED — this is the standard mechanism

for both platforms and is consistent with there being no deploy step in the CI workflow file, but the specific branch/trigger configuration on each platform's dashboard was not independently confirmed from source, since that configuration lives in the platform's own settings, not in a committed file).

15.6 Environment parity

Environment	Database	API base URL used by frontend	Notes
Local dev	Docker Compose Postgres	/api (Vite proxy, per code comment in apps/web/src/lib/api.ts)	NODE_ENV unset/development
CI	GitHub Actions postgres:16-alpine service container	N/A (frontend build only, no live API call)	NODE_ENV=test
Production	Railway-hosted Postgres	VITE_API_URL (build-time env var)	NODE_ENV=production (assumed set by Railway; not independently confirmed as a literal file/setting in this repo)

15.7 What this document cannot confirm

- The exact Railway project/service topology (single service vs. multiple, database plugin vs. external Postgres) beyond what railway.json and the Prisma provider imply. UNKNOWN in detail — Not determined from the available source code/project artefacts.
- Custom domain configuration, CDN/edge caching settings, or Vercel project-level environment variable values. UNKNOWN — these are platform dashboard settings, not committed files.
- Backup/disaster-recovery policy for the production database. UNKNOWN — Not determined from the available source code/project artefacts.
- Scaling configuration (instance count, autoscaling rules) for the Railway service. UNKNOWN — not expressed in railway.json, which only configures build/deploy/healthcheck behaviour.

16 — Configuration & Environment

All variables below are CONFIRMED directly from apps/api/src/config/env.ts (backend) and apps/web/src/lib/api.ts/apps/web/vercel.json (frontend). Actual secret values are never present in the repository and are not reproduced here; where a value is needed for illustration it is shown as [REDACTED].

16.1 Backend (apps/api) — loaded via dotenv from .env (or .env.test when NODE_ENV=test)

Variable	Required?	Default	Purpose
DATABASE_URL	Required (throws at boot if missing)	—	PostgreSQL connection string, consumed by Prisma
JWT_SECRET	Required in non-test envs (defaults to test-secret only when NODE_ENV=test)	test-secret (test only)	HMAC signing key for session JWTs
JWT_EXPIRES_IN	Optional	7d	Session token lifetime
COOKIE_NAME	Optional	event_rsvp_token	Name of the httpOnly session cookie
CORS_ORIGINS	Optional	http://localhost:5173	Comma-separated allow-list of origins for CORS
PUBLIC_APP_URL	Optional	http://localhost:5173	Base URL used to build outbound links (invite links, Stripe/PayPal return URLs)
RSVP_RATE_LIMIT_MAX	Optional	20	Max RSVP submissions per window
RSVP_RATE_LIMIT_WINDOW_MS	Optional	900000 (15 min)	RSVP rate-limit window
RESEND_API_KEY	Optional	—	Enables invite email sending via Resend; without it, invite-email endpoints

			return a clear configuration error rather than failing silently
RESEND_FROM_EMAIL	Optional (required alongside RESEND_API_KEY)	—	Verified sending address for invite emails
STRIPE_SECRET_KEY	Optional	—	Enables Stripe checkout + Stripe Connect payouts; without it, checkout/payout endpoints for Stripe return a clear configuration error
STRIPE_WEBHOOK_SECRET	Optional	—	Verifies Stripe webhook signatures (own-account events)
STRIPE_CONNECT_WEBHOOK_SECRET	Optional	—	Verifies Stripe webhook signatures for a separate connected-accounts event destination, if the Stripe dashboard was configured with two destinations instead of one covering both audiences
PAYSTACK_SECRET_KEY	Optional	—	Enables NGN payouts via Paystack Subaccounts
PAYPAL_CLIENT_ID	Optional	—	Enables PayPal checkout
PAYPAL_CLIENT_SECRET	Optional	—	Enables PayPal checkout
PAYPAL_MODE	Optional	sandbox	sandbox or live — selects PayPal API base URL
PLATFORM_FEE_	Optional	5	EventFlow's cut of

PERCENT			each merchandise sale
TICKET_FEE_PER CENT	Optional	5	EventFlow's cut of each ticket sale (independently tunable from the merchandise fee)
PORT	Optional	4000	HTTP listen port
NODE_ENV	Implicit (set by platform)	development	Controls .env vs .env.test loading, cookie secure/sameSite behaviour, and error-message verbosity

CONFIRMED — every row above is read verbatim from apps/api/src/config/env.ts.

Redacted example .env (illustrative only, not a real file)

```

DATABASE_URL=postgresql://user:[REDACTED]@host:5432/dbname
JWT_SECRET=[REDACTED]
JWT_EXPIRES_IN=7d
COOKIE_NAME=event_rsvp_token
CORS_ORIGINS=https://your-frontend.vercel.app
PUBLIC_APP_URL=https://your-frontend.vercel.app
RSVP_RATE_LIMIT_MAX=20
RSVP_RATE_LIMIT_WINDOW_MS=900000
RESEND_API_KEY=[REDACTED]
RESEND_FROM_EMAIL=invites@yourdomain.com
STRIPE_SECRET_KEY=[REDACTED]
STRIPE_WEBHOOK_SECRET=[REDACTED]
STRIPE_CONNECT_WEBHOOK_SECRET=[REDACTED]
PAYSTACK_SECRET_KEY=[REDACTED]
PAYPAL_CLIENT_ID=[REDACTED]
PAYPAL_CLIENT_SECRET=[REDACTED]
PAYPAL_MODE=live
PLATFORM_FEE_PERCENT=5
TICKET_FEE_PERCENT=5
PORT=4000
NODE_ENV=production

```

16.2 Frontend (apps/web) — Vite build-time environment variables

Variable	Required?	Default	Purpose
VITE_API_URL	Optional	/api (relative — assumes same-origin or a dev proxy)	Base URL the frontend calls for all API requests; must be set to the

deployed API's
public origin in
production

CONFIRMED — `apps/web/src/lib/api.ts` (`import.meta.env.VITE_API_URL || "/api"`). Vite embeds this at **build time**, not runtime — changing it requires a rebuild/redeploy, not just a server restart.

16.3 CI-only environment (not used in production)

CONFIRMED — `.github/workflows/ci.yml` env: block for the api job:

`DATABASE_URL` (points at the ephemeral CI Postgres container), `NODE_ENV=test`, `JWT_SECRET=ci-test-secret`, `JWT_EXPIRES_IN=1h`, `COOKIE_NAME=event_rsvp_token`, `CORS_ORIGINS=http://localhost:5173`, `PUBLIC_APP_URL=http://localhost:5173`, `RSVP_RATE_LIMIT_MAX=1000`, `RSVP_RATE_LIMIT_WINDOW_MS=900000` (both rate-limit values deliberately loosened so test runs aren't self-throttled). The web job sets only `VITE_API_URL=https://placeholder-api.example.com/api` for the build step.

16.4 Optional-feature configuration pattern

A repeated, consistent pattern across this codebase: any integration with an external paid service (Resend, Stripe, Paystack, PayPal) is **entirely optional at the environment level** — if its variables are unset, the relevant service-layer function throws a clear, user-facing `BadRequestError` explaining exactly which variables to set and where to get them (e.g. "Stripe isn't configured yet. Add `STRIPE_SECRET_KEY` (and `STRIPE_WEBHOOK_SECRET`) to enable checkout and payouts."), rather than the application failing to boot or behaving unpredictably. CONFIRMED — `apps/api/src/lib/stripeClient.ts`, `apps/api/src/lib/paystackClient.ts`, `apps/api/src/lib/paypalClient.ts`, `apps/api/src/modules/guests/invite.service.ts` (`getResendClient`) all follow this identical lazy-check-and-throw pattern.

16.5 Not found in the repository

No `.env.example` file was located during this documentation pass among the files directly read (its existence elsewhere in the repo was not exhaustively ruled out via a dedicated search in this pass) — Not determined from the available source code/project artefacts whether one exists; if present, it would be the canonical illustrative reference and should be reconciled against §16.1 above.

17 — Dependencies & Technology Stack

All version numbers below are CONFIRMED directly from apps/api/package.json and apps/web/package.json (as caret ranges ^x.y.z, i.e. the minimum version pinned; the exact resolved version installed depends on package-lock.json, which was not read in this pass).

17.1 Backend (apps/api) — runtime dependencies

Package	Version	Role
@prisma/client	^5.20.0	ORM client
express	^4.19.2	HTTP framework
express-async-errors	^3.1.1	Allows async route handlers to throw without manual try/catch
express-rate-limit	^7.4.0	Rate limiting middleware
helmet	^7.1.0	Security headers
cors	^2.8.5	CORS handling
cookie-parser	^1.4.6	Cookie parsing
jsonwebtoken	^9.0.2	JWT sign/verify
bcryptjs	^2.4.3	Password hashing
zod	^3.23.8	Schema validation
multer	^1.4.5-lts.1	Multipart/file upload handling
dotenv	^16.4.5	Environment variable loading
stripe	^17.3.1	Stripe SDK (checkout + Connect)
qrcode	^1.5.4	QR code generation (invites)
resend	^4.0.1	Transactional email
pdfkit	^0.15.0	Server-side PDF generation (exports)
csv-parse / csv-stringify	^5.5.6 / ^6.5.1	CSV import/export
pino / pino-http	^9.4.0 / ^10.3.0	Structured logging
morgan	^1.10.0	HTTP request logging (present alongside pino-http — INFERRED both may not be active simultaneously in app.ts,

nanoid ^3.3.7

which was confirmed to use pino-http; morgan's actual usage site was not located in the files read this pass)

Short unique ID generation (specific usage site not confirmed this pass — Prisma's own cuid() is used for model IDs)

17.2 Backend — development dependencies

Package

typescript

tsx

prisma

vitest

supertest

embedded-postgres

eslint + @typescript-eslint/*

pino-pretty

@types/*

17.3 Frontend (apps/web) — runtime dependencies

Package	Version	Role
react / react-dom	^18.3.1	UI framework
react-router-dom	^6.26.2	Client-side routing
@tanstack/react-query	^5.59.0	Server-state data fetching/caching
axios	^1.7.7	HTTP client
react-hook-form	^7.53.0	Form state management
@hookform/resolvers	^3.9.0	Zod↔react-hook-form integration
zod	^3.23.8	Shared schema validation (client-side mirror of backend schemas — INFERRED shared validation intent, not confirmed as literally shared schema files)
konva / react-konva	^9.3.22 / ^18.2.16	Canvas rendering for the

jsqr	^1.4.0	seating planner QR code decoding (camera-based scan mode)
jspdf / jspdf-autotable	^2.5.2 / ^3.8.4	Client-side PDF generation
xlsx	^0.18.5	Spreadsheet export
sonner	^1.5.0	Toast notifications
lucide-react	^0.446.0	Icon set
clsx / tailwind-merge	^2.1.1 / ^2.5.2	Conditional/merged className utilities
@fontsource/dm-sans / @fontsource/outfit	^5.3.0	Self-hosted webfonts

17.4 Frontend — development dependencies

Package	Version	Role
vite	^5.4.8	Build tool / dev server
@vitejs/plugin-react	^4.3.1	React fast-refresh plugin for Vite
typescript	^5.6.2	Language/compiler
tailwindcss	^3.4.13	Utility-first CSS
postcss / autoprefixer	^8.4.47 / ^10.4.20	CSS processing pipeline for Tailwind
eslint + @typescript-eslint/* + eslint-plugin-react-hooks + eslint-plugin-react-refresh	various	Linting

No test framework, state-management library beyond React Query/component state, or CSS-in-JS library is present in the frontend dependency list. CONFIRMED by absence.

17.5 Technology stack summary

Layer	Technology
Backend language/runtime	TypeScript (compiled to CommonJS, apps/api/package.json "type": "commonjs"), Node.js ≥20 (root package.json engines.node)
Backend framework	Express 4
ORM	Prisma 5
Database	PostgreSQL 16 (per docker-compose.yml/CI image; the exact production version is managed by Railway and not independently confirmed)

Frontend language	TypeScript, ESM (apps/web/package.json "type": "module")
Frontend framework	React 18
Build tool	Vite 5
Styling	Tailwind CSS 3
Package management	npm workspaces (root package.json), no lockfile format other than npm's own confirmed in this pass
CI	GitHub Actions
Hosting	Railway (API), Vercel (frontend)
Payments	Stripe / Stripe Connect, Paystack, PayPal
Email	Resend

17.6 Not assessed in this pass

This document is an inventory, not a vulnerability audit. Whether any listed dependency has a known CVE at the current pinned/resolved version was not checked (no npm audit or equivalent was run as part of this documentation effort — see 14-security-documentation.md §14.3). License compliance of third-party packages was also not reviewed. Both are UNKNOWN — Not determined from the available source code/project artefacts.

18 — User Guide (Planner)

This guide describes how to use EventFlow as a planner, based on the capabilities confirmed to exist in the application (see 02-functional-requirements.md for the underlying evidence). Exact button labels/wording were not re-verified against every screen in this pass — treat step-by-step wording as INFERRED from route/component names and prior UI review, not a verbatim transcript of on-screen copy.

18.1 Getting started

1. Go to the EventFlow site and select **Register**. Provide your name, email, and a password (at least 8 characters, containing a letter and a number).
2. You're automatically signed in after registering and taken to **My Events**.
3. To sign back in later, use **Log in** with the same email/password. There is currently no self-service "forgot password" flow — if you're locked out, you'll need to contact support (see 20-known-issues-risks-technical-debt.md).

18.2 Creating and managing an event

1. From **My Events**, select **Create Event**. Provide a name, event type, and date at minimum; venue, capacity, an RSVP deadline, a cover image, and a custom welcome message are all optional.
2. Decide which RSVP form fields you want to collect — plus-ones, plus-one names, meal selection, dietary requirements, accessibility needs, and special requests can each be turned on or off per event.
3. Once created, your event gets its own private, shareable RSVP link automatically — you don't need to generate anything for the basic shared-link flow to work.
4. Edit event details at any time from the event's top bar (**Edit**). Deleting an event is permanent and only available to the event's owner.

18.3 Adding and inviting guests

1. Open the **Guests** tab. Add guests one at a time, or import a spreadsheet via **Import CSV**.
2. Each guest can have plus-ones tracked either as a simple count or as named individuals (if you've enabled "plus-one names" on the event).
3. For a personal touch, open a guest's **Invite** panel to get a QR code and link unique to them — their RSVP form will already know their name. You can send this by email directly from EventFlow (if your account has email sending configured) or copy the link/QR for WhatsApp or any other channel.
4. If you've designed your own invitation card (a PDF or image), upload it once on the **RSVP** tab — it will be attached automatically to every invite email sent from then on.
5. Track who hasn't responded from the **RSVP** tab's non-responder list, so you know who to follow up with before your deadline.

18.4 Seating planner

1. Open the **Seating** tab. Add tables by shape and capacity — seats are created automatically.
2. Drag a confirmed guest from the sidebar onto a table. Their plus-ones (if named) are seated at nearby seats automatically.
3. If a table is full, EventFlow will warn you rather than silently blocking the move in most cases — you can choose to seat someone anyway if you know you'll add an extra chair.
4. If you shrink a table's capacity below what's currently seated, EventFlow tells you exactly who was unseated so you can reseal them.
5. Export a printable seating chart as a PDF from the Seating tab.

18.5 Vendors

1. Open the **Vendors** tab and add each vendor with their category, status, contact details, and agreed cost (choose the currency you're paying them in).
2. As you move a vendor's status forward (contacted → quote received → booked → confirmed), you'll get a notification confirming the change was saved.
3. If you're paying vendors in more than one currency, your totals are shown per currency, not blended into one misleading number.

18.6 Selling merchandise

1. Open the **Merchandise** tab and add products with a price, currency, and optional stock limit.
2. Before guests can buy anything, turn on the shop for your event and connect at least one way to get paid in that currency (see §18.7).
3. Once live, guests see your shop directly on their RSVP page and can buy with any connected, matching-currency payment method.

18.7 Getting paid (Payouts)

1. From the Merchandise or Tickets tab, open **Payouts**.
2. For USD or GBP, connect **Stripe** — you'll be sent to Stripe's own onboarding to enter your business/bank details; EventFlow never sees or stores them.
3. For NGN, connect **Paystack** with your Nigerian bank account and account number.
4. For any currency, you can also connect **PayPal** with just your PayPal email as an additional option alongside Stripe/Paystack.
5. A small platform fee (currently 5%, the same for merchandise and tickets) is taken automatically from each sale — you receive the rest directly from the processor.

18.8 Selling tickets to a public event

1. Make your event public from the **Tickets** tab's publish settings — this gives it a public page anyone can find and buy from, without needing an invite.
2. Add one or more ticket types (General Admission, VIP, Early Bird, etc.) with a price, currency, and optional total quantity and sales window.

3. Share your event's public ticket page link — no login required for buyers.
4. On the day, use the **Check-in** tab's QR scanner to scan each ticket at the door. Already-used tickets are flagged instead of silently accepted twice.

18.9 Check-in day

1. Open the **Check-in** tab. You can scan a guest's or ticket buyer's QR code with a device camera, or search and check someone in manually if a scan fails.
2. Check-in status updates live across the app — your dashboard and guest list reflect it immediately.

18.10 Notifications and Analytics

- The bell icon in the top bar shows recent activity — new RSVPs, vendor status changes, and paid orders — across all your events.
- The **Analytics** page (top nav) gives you a cross-event view: RSVP funnel, check-in rate, and vendor spend by currency, across every event you own.

19 — Administrator Guide

This guide covers the ADMIN role’s capabilities (internal EventFlow support/staff use), based on `apps/api/src/modules/admin/*` and the corresponding frontend Admin section. See `10-roles-and-permissions.md` for the full permission matrix.

19.1 Becoming an admin

There is no self-service way to become an admin — role cannot be set via registration. UNKNOWN — Not determined from the available source code/project artefacts exactly how the first admin account is provisioned (likely a direct database update or a seed script referenced by `apps/api/package.json`’s seed script, whose contents were not read in this pass). Once your account’s role is ADMIN in the database, the **Admin** link appears in the top navigation automatically.

19.2 Subscriber and event support

1. Open **Admin** → the subscriber/event browser lists every user and every event on the platform, with counts (events per subscriber, guests/orders per event).
2. To help a specific subscriber, open their event directly — you land on the exact same Overview/Guests/RSVP/Seating/Vendors/Merchandise/Tickets/Check-in screens the subscriber themselves would see, with full read/write access, identified by a “Support view — editing as admin” badge.
3. Two things you explicitly **cannot** do from support mode, even though everything else is available: delete the subscriber’s event, or connect/change their payout account (Stripe/Paystack/PayPal) details. These remain the subscriber’s own responsibility.
4. Every change you make on a subscriber’s event (not reads) is recorded automatically — you don’t need to log anything yourself.

19.3 Audit log

- **Admin** → **Audit Log** shows every recorded admin action: who, on which event, what HTTP method, and a plain-language summary (e.g. “Updated guest,” “Created vendor”).
- Filterable by event and by admin user.
- Only mutations on events you don’t own are logged — your own events (if you also use EventFlow as a planner) and read-only actions on subscribers’ events are not recorded here, since there’s nothing anomalous to flag in either case.
- Note: audit log writes are best-effort; a failure to write a log entry does not block or roll back the underlying action, and is not currently surfaced anywhere if it happens.

19.4 Payment oversight

- **Admin** → **Payment Events** (or the per-event equivalent when viewing a subscriber’s event) shows every payment attempt any processor has reported — successes, declines, failures, and expirations — not just completed orders. Use this to investigate

“I paid but it shows unpaid”-type disputes: the raw provider payload is retained for each entry.

- Filterable by event, order, status, and provider.

19.5 Platform analytics

Admin → Analytics shows: total subscribers, total events, total guests, RSVP confirmation rate, total paid orders, revenue broken out by currency and by provider (never blended into one number), and a 30-day trend of new signups and events created.

19.6 Managing marketing content

- **Admin → Articles:** write, edit, publish/unpublish, and delete blog posts. A URL slug is generated automatically from the title the first time you save and never changes afterward — even if you edit the title later, existing shared links to the article keep working.
- **Admin → Services:** add, edit, reorder (drag), hide, or delete the “what we offer” cards shown on the public landing page. Icons are chosen from a fixed set, not uploaded — this is by design, not a current limitation to be worked around.

19.7 Operational notes

- Admin access is enforced server-side on every request (requireAdmin performs a fresh database role lookup), not just hidden in the UI — a non-admin cannot reach admin functionality by guessing a URL.
- There is currently no in-app way to promote another user to ADMIN or to view a full list of who else has admin access — user role management, if needed, happens outside the application (database-level), based on the routes reviewed in this pass.

20 — Known Issues, Risks & Technical Debt

Severity is a judgment call applied during this documentation pass based on observed impact, not a rating taken from an existing issue tracker (no issue tracker was reviewed as part of this effort — GitHub Issues, if any, were not read). Each item is marked **Confirmed** (directly observed in code/config) or **Potential** (a risk inferred from an absence or pattern, not directly proven).

Critical

None identified as Critical from static review alone. Any item that would warrant this severity (e.g. an authentication bypass or unrestricted financial mutation) would require dynamic testing to confirm exploitability, which was explicitly out of scope for this pack — see 14-security-documentation.md.

High

#	Issue	Status	Evidence	Impact
H-1	No automated test coverage for the public ticket checkout flow (capacity reservation under concurrency, provider routing, PayPal capture, ticket issuance)	Confirmed	No file matching this scope exists among apps/api/tests/** (see 12-testing-documentation.md §12.3)	This is the payment-and-inventory-critical path for the ticketing product line; regressions here (e.g. an oversold ticket type, or a fee miscalculation) would not be caught by CI.
H-2	No self-service password reset / account recovery flow	Confirmed	Only 4 auth routes exist (register, login, logout, me) — apps/api/src/modules/auth/auth.routes.ts	A locked-out planner has no in-app recovery path; support must intervene manually, and the mechanism for doing so is itself undocumented in the code reviewed.
H-3	JWT sessions	Confirmed	apps/api/src/	A token

are not
revocable;
“logout” only
clears the client
cookie

modules/
auth/
auth.controller.
r.ts (logout),
apps/api/src/u
tils/jwt.ts (no
blocklist)

captured before
logout remains
valid for up to 7
days (the
default
JWT_EXPIRES
_IN) regardless
of the user
logging out.

Medium

#	Issue	Status	Evidence	Impact
M-1	RSVPQuestion/RSVPAnswer models exist in the schema with no corresponding API routes	Confirmed	apps/api/prisma/schema.prisma; no route file among the 22 reviewed exposes CRUD for either model	Dead/scaffolded schema surface — either an incomplete feature or leftover design exploration; adds schema complexity with no current functional benefit, and could confuse future maintainers about whether custom RSVP questions are supported.
M-2	AdminAuditLog writes are fire-and-forget with silently swallowed failures	Confirmed	apps/api/src/middleware/auditAdminEventActions.ts (.catch(() => undefined))	An admin support action’s audit trail entry can silently fail to be written, with no alerting — undermines the accountability goal the audit log exists for, in the specific case where the write

M-3	Database migration failures on deploy do not block the API server from starting	Confirmed	apps/api/railway.json (startCommand runs node dist/index.js unconditionally after the migration step, regardless of its exit code)	itself fails. A failed migration could leave the running application out of sync with the actual database schema, likely surfacing as confusing runtime errors rather than a clearly failed deployment.
M-4	No CSRF-specific mitigation identified for cookie-authenticated requests beyond CORS origin restriction	Potential	Absence of any CSRF middleware in apps/api/src/middleware/*; sameSite: "none" cookie in production	Whether this is an actual exploitable gap depends on browser same-site cookie enforcement and the CORS allow-list holding correctly in all deployed environments — not verified by testing.
M-5	CSV guest-import row-level validation/error-handling behaviour is undocumented from source read in this pass	Potential	Controller-level CSV parsing logic in guests.controller.ts was not read line-by-line this pass	Unknown whether a malformed row fails the whole import batch, is silently skipped, or is rejected with a clear per-row error — affects planner experience and data quality; see 24-documentation-

M-6	No account lockout after repeated failed login attempts, beyond a shared IP-based rate limit	Confirmed (absence)	apps/api/src/modules/auth/auth.routes.ts (only a 30/15min IP-keyed limiter)	gap-analysis.md. A distributed brute-force attempt (many source IPs) against a single account is not mitigated by the current limiter design.
-----	--	-------------------------------	---	--

Low

#	Issue	Status	Evidence	Impact
L-1	Both pino-http and morgan are listed as backend dependencies, but only pino-http was observed wired into app.ts	Potential	apps/api/package.json lists both; apps/api/src/app.ts only imports/uses pino-http	Likely an unused dependency left over from an earlier logging setup — minor bundle/attack-surface bloat, not a functional issue.
L-2	nanoid is a listed dependency with no confirmed usage site in the files read this pass (Prisma's own cuid() is used for all model IDs)	Potential	apps/api/package.json; no import ... from "nanoid" encountered across the modules read	Possibly unused, or used in a file not reviewed this pass (e.g. a script or a module not covered) — flagged for verification rather than asserted as certainly dead.
L-3	No coverage-percentage enforcement, mutation testing, or accessibility (a11y)	Confirmed (absence)	No such tooling in apps/api/package.json / apps/web/package.json / ci.yml	Standard maturity gap for a project at this stage; not urgent but worth planning for as the

	automated testing			codebase grows.
L-4	No automated end-to-end (browser-driven) test suite	Confirmed (absence)	No Playwright/Cypress/similar dependency in either package.json	Multi-step user journeys (e.g. full guest RSVP → seating → check-in) are only verified manually or via the API-level integration tests, not through the actual UI.
L-5	Frontend has zero automated test coverage of any kind	Confirmed (absence)	No test script/dependency in apps/web/package.json	All frontend correctness currently depends on TypeScript's type checking, linting, and manual verification.

Architectural notes (not defects, but worth surfacing)

- The event dashboard (getEventDashboard) issues roughly 15 separate database queries per load (13 in parallel, 2 sequential follow-ups) — apps/api/src/modules/events/events.service.ts. Not confirmed to be a performance problem (no load testing was performed), but is the kind of pattern worth watching as guest-list sizes grow. **Potential**, not Confirmed as an actual bottleneck.
- The “admin bypass via the owner-scoped endpoints” architecture (see 10-roles-and-permissions.md §10.3) is an elegant way to avoid a parallel admin API, but it does mean every new owner-scoped endpoint added in the future automatically becomes admin-accessible unless a developer remembers to explicitly exclude it (as was done for event deletion and payout connection) — a process/discipline risk rather than a current code defect.

Explicitly out of scope for this list

Business/product-strategy gaps (e.g. “should EventFlow support a fourth payment processor,” “should there be a mobile native app”) are not technical debt and are not listed here — this document covers implementation-level issues only, per the brief.

21 — Traceability Matrix

Maps each functional requirement (02-functional-requirements.md) to its implementing code and its automated test coverage (from 12-testing-documentation.md). “Test file” lists the integration/unit test file that covers the module the requirement belongs to — this is coverage at the module level, not a claim that every individual requirement has its own dedicated test case (verifying that level of granularity would require reading every assertion in every test file, which was not done in this pass).

Req ID	Requirement (short)	Implementing code	Test file
FR-001	Register with validated email/password	auth.service.ts, auth.schema.ts	auth.test.ts
FR-002	Reject duplicate email	auth.service.ts	auth.test.ts
FR-003	Bcrypt password hashing	utils/password.ts	No dedicated test found
FR-004	Login issues JWT cookie	auth.controller.ts	auth.test.ts
FR-005	Logout clears cookie	auth.controller.ts	auth.test.ts
FR-006	GET /auth/me	auth.routes.ts	auth.test.ts
FR-007	Auth rate limiting	auth.routes.ts	No dedicated test found
FR-010	Create event	events.service.ts	events.test.ts
FR-011	Unique rsvpToken per event	schema.prisma	events.test.ts (implicit)
FR-012	Owner/admin update, owner-only delete	events.service.ts (getOwnedEvent/getOwnedEventStrict)	events.test.ts, admin.test.ts
FR-013	Cover image upload	events.service.ts	No dedicated test found
FR-014	Event dashboard stats	events.service.ts	events.test.ts
FR-015	Events list with guest summary	events.service.ts	events.test.ts
FR-016	isPublic + immutable publicSlug	events.service.ts	No dedicated test found
FR-020	Guest CRUD + party	guests.service.ts	guests.test.ts

	members		
FR-021	Guest list search/filter	guests.service.ts	guests.test.ts
FR-022	CSV import	guests.routes.ts, guests.service.ts	guests.test.ts (extent of row-level coverage unconfirmed — see M-5 in doc 20)
FR-023	CSV/PDF/wristband export	guests.routes.ts	guests.test.ts (extent unconfirmed)
FR-024	Manual + QR check-in	guests.service.ts, invite.service.ts	guests.test.ts
FR-025	Auto seat release on RSVP change	utils/rsvpMath.ts, guests.service.ts	rsvpMath.test.ts
FR-030	Public RSVP page by token	rsvp.service.ts	rsvp.test.ts
FR-031	RSVP submission (shared + invite token)	rsvp.service.ts	rsvp.test.ts
FR-032	Deadline/closed enforcement	rsvp.service.ts	rsvp.test.ts
FR-033	RSVP rate limiting	rsvp.routes.ts	No dedicated test found
FR-034	Owner notification on RSVP change	notifications.service.ts	notifications.test.ts, rsvp.test.ts (indirect)
FR-040	Personalised invite link + QR	invite.service.ts	guests.test.ts (extent unconfirmed)
FR-041	Invite email via Resend	invite.service.ts	inviteEmail.test.ts (content generation only — actual send not integration-tested per §12.5)
FR-042	Bulk invite email send	invite.service.ts	No dedicated test found
FR-043	Invitation card upload/manage	invitationCard.service.ts	invitationCard.test.ts
FR-050	Venue layout + decor objects	seating.service.ts	seating.test.ts
FR-051	Table creation with	seating.service.ts	seating.test.ts

	auto seats		
FR-052	Guest + party assignment with capacity rules	seating.service.ts, utils/capacity.ts	seating.test.ts, capacity.test.ts
FR-053	Capacity-shrink unseating	seating.service.ts	seating.test.ts
FR-054	Unassign guest / party member	seating.service.ts	seating.test.ts
FR-055	Seating map PDF export	seating.routes.ts	No dedicated test found
FR-060	Vendor CRUD	vendors.service.ts	vendors.test.ts
FR-061	Per-currency vendor cost totals	vendors.service.ts	vendors.test.ts
FR-062	Vendor status-change notification	notifications.service.ts	notifications.test.ts, vendors.test.ts (indirect)
FR-070	Product CRUD	products.service.ts	merchandise.test.ts
FR-071	Shop enabled toggle	products.service.ts	merchandise.test.ts
FR-072	Guest checkout	orders.service.ts	merchandise.test.ts
FR-073	Reject mixed-currency cart	orders.service.ts	merchandise.test.ts (extent unconfirmed)
FR-074	Stock decrement only on PAID	orders.service.ts	merchandise.test.ts
FR-080	Connect Stripe/Paystack/PayPal payout accounts	payouts.service.ts	payouts.test.ts
FR-081	Default provider preference routing	orders.service.ts	merchandise.test.ts (extent unconfirmed)
FR-082	Platform fee capture, frozen on order	orders.service.ts, schema.prisma	merchandise.test.ts
FR-083	Webhook signature verification	orders.service.ts	merchandise.test.ts (signature-failure path specifically unconfirmed — see §12.5)

FR-084	Payment event logging	orders.service.ts	merchandise.test.ts (extent unconfirmed)
FR-090	Ticket type CRUD	ticketTypes.service.ts	ticketTypes.test.ts
FR-091	Public ticket listing page	checkout.service.ts (tickets)	No test found
FR-092	Atomic capacity reservation	ticketTypes.service.ts (reserveTickets)	No test found (see H-1, doc 20)
FR-093	Capacity release on failed/expired order	orders.service.ts (releasePendingTicketOrder)	No test found
FR-094	Ticket issuance on PAID	orders.service.ts (finalizeOrderPaid)	No test found
FR-095	Door check-in by ticket code	ticketTypes.service.ts (checkInTicketByCode)	Likely covered by ticketTypes.test.ts (co-located route) — not individually confirmed
FR-100	Notification generation (RSVP/vendor/order)	notifications.service.ts	notifications.test.ts
FR-101	Mark read / mark all read	notifications.routes.ts	notifications.test.ts
FR-102	“Needs Attention” insights	insights.service.ts	insights.test.ts
FR-110	Planner cross-event analytics	analytics.service.ts	analytics.test.ts
FR-111	Platform-wide analytics	admin.service.ts	admin.test.ts
FR-120	Admin-only route gating	middleware/auth.ts (requireAdmin)	admin.test.ts
FR-121	List all subscribers/events	admin.service.ts	admin.test.ts
FR-122	Admin event drill-in via owner-scoped endpoints	events.service.ts (getOwnedEvent)	admin.test.ts (extent of drill-in-specific assertions unconfirmed)
FR-123	Audit log write on	middleware/	admin.test.ts

	admin mutation	auditAdminEventActions.ts	(extent unconfirmed)
FR-124	Admin blocked from delete/payout-connect	events.service.ts, payouts.service.ts	No dedicated test found
FR-130	Landing page + FAQ + Services	landing.service.ts, frontend LandingPage.tsx	landingServices.test.ts (backend only)
FR-131	Article authoring/publish workflow	articles.service.ts	articles.test.ts
FR-132	Public article reads	articles.public.routes.ts	articles.test.ts
FR-140	/health endpoint	app.ts	No dedicated test found
FR-141	PWA installability	apps/web/public/manifest.webmanifest, sw.js, InstallPrompt.tsx	N/A — no frontend test infrastructure
FR-142	Security headers + CORS allow-list	app.ts	No dedicated test found

Summary

Of 68 functional requirements listed in 02-functional-requirements.md, the majority trace to at least module-level integration test coverage. The clearest, most consequential coverage gap is the public ticket checkout/capacity-reservation/issuance chain (FR-091 through FR-094), consistent with the standalone finding in 12-testing-documentation.md §12.3 and 20-known-issues-risks-technical-debt.md H-1.

22 — Application Inventory

All counts CONFIRMED via direct file listing (Glob) during this documentation pass, current as of the point-in-time this pack was prepared.

22.1 Backend (apps/api)

Item	Count	Method
Express route files (*routes.ts)	22	Glob apps/api/src/modules/**/* .routes.ts
API endpoints (all HTTP method+path combinations)	122	Manual enumeration of every route table in 09-api- documentation.md §9.1– 9.15
Service files (*.service.ts)	18	Glob apps/api/src/modules/**/* .service.ts
Prisma models	27	Direct count of model blocks in apps/api/prisma/schema. prisma
Prisma enums	14	UserRole, EventType, PublicEventCategory, RsvpStatus, LayoutObjectType, TableShape, VendorCategory, VendorStatus, NotificationType, Currency, PayoutProvider, TicketStatus, PaymentEventStatus, OrderStatus, OrderKind, ArticleStatus — (<i>recount: 16 enums; see note below</i>)
Middleware files	4	auth.ts, validate.ts, errorHandler.ts, auditAdminEventActions. ts
Shared lib/ files	5	prisma.ts, logger.ts, errors.ts, apiResponse.ts, stripeClient.ts,

		paystackClient.ts, paypalClient.ts (7 — <i>see note below</i>)
Shared utils/ files	5	password.ts, jwt.ts, rsvpMath.ts, capacity.ts, email.ts
Test files (unit + integration + helpers + setup)	22	Glob apps/api/tests/**/*.*ts
— of which unit tests	3	rsvpMath.test.ts, capacity.test.ts, inviteEmail.test.ts
— of which integration tests	16	See 12-testing- documentation.md §12.2
— of which helpers/setup	3	authHelpers.ts, testApp.ts, globalSetup.ts

Note on enum/lib counts: A manual recount while drafting this table found 16 Prisma enums (the list above enumerates all 16) and 7 files under apps/api/src/lib/ (prisma.ts, logger.ts, errors.ts, apiResponse.ts, stripeClient.ts, paystackClient.ts, paypalClient.ts) rather than the initially-stated 5 — corrected inline rather than silently reconciled, per this pack’s evidence-honesty requirement: an earlier draft pass undercounted both figures before a second pass caught the discrepancy.

22.2 Frontend (apps/web)

Item	Count	Method
Page-level components (src/pages/**/*.*tsx)	37	Glob apps/web/src/pages/**/*.*ts sx
Data-fetching hooks (src/hooks/*.*ts)	17	Glob apps/web/src/hooks/*.*ts
Shared UI/layout/marketing/illustr ation components (src/components/**/*.*tsx)	31	Glob apps/web/src/components /**/*.*tsx
Top-level routes defined in App.tsx	20	Direct count of <Route> elements in apps/web/src/App.tsx (including the root redirect and catch-all)
Test files	0	Confirmed absence — no test script/dependency in apps/web/package.json

Page inventory by area

- **Marketing** (3): LandingPage, ArticlesListPage, ArticleDetailPage (shared marketing components SiteHeader, SiteFooter, HeroIllustration are counted separately under components, not here).
- **Auth** (2): LoginPage, RegisterPage.
- **Public RSVP/shop** (2): PublicRsvpPage, ShopSection.
- **Public tickets** (1): PublicTicketEventPage.
- **Events — planner** (13): EventsListPage, EventFormModal, OverviewTab, GuestsTab, GuestFormModal, CsvImportModal, InviteModal, RsvpTab, VendorsTab, VendorFormModal, MerchandiseTab, ProductFormModal, PayoutsSection.
- **Events — tickets** (3): TicketsTab, TicketTypeFormModal, PublishSettingsPanel.
- **Events — seating** (5): SeatingTab, SeatingCanvas, AddTableModal, SelectionPanel, GuestSidebar.
- **Events — check-in** (2): CheckInTab, TicketScanPanel.
- **Events — routing glue** (1): EventTabPage.
- **Analytics** (1): AnalyticsPage.
- **Admin** (5): AdminPage, PlatformAnalyticsTab, ArticlesTab, ServicesTab, ArticleFormModal, ServiceFormModal (6 — *corrected count*).

Totals above are drawn directly from the Glob file listing performed during this pass; category subtotals are grouped manually by directory/purpose and may not sum to exactly 37 due to shared modal components appearing under more than one natural category — treat category subtotals as organizational, and the top-line count of 37 as the authoritative figure.

22.3 Configuration & infrastructure files

File

apps/api/railway.json
apps/web/vercel.json
.github/workflows/ci.yml
docker-compose.yml
apps/api/prisma/schema.prisma
Root package.json
apps/api/package.json, apps/web/package.json

22.4 Third-party service integrations

Stripe (+ Stripe Connect), Paystack, PayPal, Resend — 4 external paid/transactional services, all confirmed via dedicated client wrapper files under apps/api/src/lib/ or direct SDK usage, and all following the same “optional, clear-error-if-unconfigured” pattern (see 16-configuration-and-environment.md §16.4).

23 — Glossary

Terms as used specifically within the EventFlow codebase and this documentation pack. CONFIRMED entries cite the schema/code source of the term; a small number of general industry terms are included for reader convenience and marked accordingly.

Term	Definition
Planner	A registered subscriber account (UserRole.PLANNER), the primary paying user who creates and manages events.
Admin	An internal EventFlow support role (UserRole.ADMIN) with cross-subscriber visibility and owner-equivalent access to any event, minus deletion and payout-connection rights.
Guest	A person invited to a private event, represented by a Guest row; may or may not have a user account (guests never log in).
RSVP token	The unique, unguessable value on Event.rsvpToken that serves as the bearer credential for the shared public RSVP link.
Invitation token	The unique value on EventInvitation.token, a personalised (per-guest) bearer credential for a pre-filled RSVP link.
Public slug	Event.publicSlug — an immutable, URL-safe identifier generated once an event is made public, used in /tickets/:slug.
isPublic	The Event boolean flag that switches an event from the private RSVP flow into the public ticket-selling flow.
Party / party member	A named individual accompanying a primary guest (a “+1”), stored as a GuestParty row, distinct from the simple numeric additionalGuestsCount.
Seating assignment	A SeatingAssignment (for a Guest) or PartySeatingAssignment (for a GuestParty member) row linking a person to a specific table and, optionally, a specific seat.

Layout object	A freeform decor element (stage, bar, dance floor, etc.) placed on the venue canvas, represented by LayoutObject.
Vendor	A third-party supplier (caterer, photographer, etc.) tracked per event via the Vendor model.
Product	A sellable merchandise item on an event's shop (Product model).
Order	A purchase (merchandise or ticket, distinguished by Order.kind) moving through PENDING → PAID/CANCELLED.
OrderKind	The enum (MERCHANDISE TICKET) distinguishing what an Order is for, allowing one checkout/payment pipeline to serve both.
Payout account	An EventPayoutAccount row: one planner's connection to one payment processor for one currency.
Platform fee	EventFlow's percentage cut of a paid order, taken via the processor's own fee mechanism, configured via PLATFORM_FEE_PERCENT (merchandise) or TICKET_FEE_PERCENT (tickets).
Stripe Connect	Stripe's platform-payments product, used here (Express accounts) to let a planner receive USD/GBP payments directly, minus EventFlow's application fee.
Paystack Subaccount	Paystack's split-payment mechanism, used here for NGN payouts.
Ticket type	A purchasable tier on a public event (e.g. "General Admission"), represented by TicketType, with its own price, currency, and capacity.
Ticket	One issued, individually-scannable admission credential (Ticket model, unique code), created per unit purchased once an order is PAID.
Check-in	The act of marking a guest (Guest.checkedIn) or ticket (Ticket.status = CHECKED_IN) as arrived at the event, via manual action or

	QR scan.
Needs Attention insight	A computed-on-demand item (Insight, no persisted table) flagging something needing planner action (e.g. an unassigned VIP), generated by the insights module.
Notification	A persisted, in-app alert (Notification model) generated as a side effect of RSVP changes, vendor status changes, or paid orders.
Admin bypass	The mechanism by which <code>getOwnedEvent</code> grants an ADMIN user the same access to an event that its owner has, without a separate admin-only API.
Admin audit log	<code>AdminAuditLog</code> — a record of every mutation an admin makes on an event they don't own, written by the <code>auditAdminEventActions</code> middleware.
Payment event	<code>PaymentEvent</code> — a log row for every payment-processor callback received (success, failure, or expiry), independent of the resulting order status.
Webhook	An inbound HTTP callback from Stripe or Paystack notifying the API of a payment outcome, verified by cryptographic signature before being trusted.
CUID	“Collision-resistant Unique IDentifier” — the ID generation strategy (<code>cuid()</code>) used for every Prisma model's primary key in this schema. <i>(General term, included for reader convenience.)</i>
Zod	The TypeScript-first schema validation library used for all request body/query/params validation in apps/api. <i>(General term.)</i>
Prisma	The TypeScript ORM used to access PostgreSQL from apps/api. <i>(General term.)</i>
PWA	Progressive Web App — the installable-web-app capability EventFlow's frontend provides via a manifest and service worker. <i>(General term.)</i>

24 — Documentation Gap Analysis

This document lists everything this pack could **not** establish from the repository, and why — the honest ledger the “Do Not Hallucinate” instruction requires. Anything not listed here that appears elsewhere in the pack as UNKNOWN is also cross-referenced below for completeness where material.

24.1 Items not verifiable from source at all

Topic	Why unavailable	Where else referenced
Business metrics (subscriber count, revenue, usage volume)	Not represented in source code; would require production database/analytics access	01-application-overview.md §1.5
Real incident/outage history, uptime	Not represented in a static repository	15-deployment-and-infrastructure.md §15.7
Actual production environment variable values	Correctly excluded from the repository entirely	16-configuration-and-environment.md
Railway/Vercel dashboard-level settings (domains, scaling, branch triggers, project topology)	Platform settings, not committed files	15-deployment-and-infrastructure.md §15.7
GitHub repository settings (branch protection, required reviews, Dependabot)	Platform settings, not expressible in a workflow YAML	12-testing-documentation.md §12.6, 13-test-automation-documentation.md §13.5
Dependency CVE/vulnerability status at time of reading	Requires running npm audit or an external scanner; not performed	17-dependencies-and-technology-stack.md §17.6
First-admin-account provisioning mechanism	apps/api/prisma/seed.ts (referenced by the seed npm script) was not read in this pass	10-roles-and-permissions.md §10.1, 19-administrator-guide.md §19.1
Runtime/dynamic security properties (timing attacks, session fixation, actual CSRF exploitability)	Requires dynamic testing, explicitly out of scope	14-security-documentation.md §14.3

24.2 Items partially read — depth limited by effort, not principle

These files/areas exist and were confirmed to exist, but were not read exhaustively line-by-line in this documentation pass; conclusions drawn about them elsewhere in this pack are

therefore based on file/route naming, adjacent code, or partial reads, and are labelled INFERRED rather than CONFIRMED where they appear.

- **Controller files** (*.controller.ts) across most modules — routes and services were read directly (routes define the exact path/method/middleware; services define the business logic), but the thin controller layer connecting them was not individually read for every module. Controllers in this codebase are consistently thin (route → controller → service, per the pattern observed in auth.controller.ts, which *was* read in full), so this is assessed as low-risk, but is disclosed rather than assumed away.
- **Frontend page component internals** for most of the 37 pages listed in 22-application-inventory.md — exact on-screen field lists, validation messages, and interaction details were not re-verified against source in this specific documentation pass for every screen (a subset — DashboardLayout.tsx, NotificationBell.tsx, UserMenu.tsx, ProtectedRoute.tsx, HeroIllustration.tsx — were read in full; others draw on this project’s broader prior UI work this session plus route/hook naming). See 06-ui-ux-specification.md §6.7.
- **apps/api/tests/** assertion-level content** — every test file’s existence and general scope (from its filename and the module it’s named after) is confirmed, but individual it()/describe() blocks inside each file were not enumerated. Coverage statements in 12-testing-documentation.md are therefore accurate at the “a test file for this module exists” granularity, not the “this exact scenario is asserted” granularity, except where a specific scenario is called out as confirmed absent (the ticket-checkout gap, which is a file-level absence and therefore fully confirmable).
- **CSV import row-level handling** (guests.controller.ts) — flagged directly in 20-known-issues-risks-technical-debt.md M-5 as a gap in this pass’s review depth, not a confirmed defect.
- **apps/api/prisma/migrations/** — migration file count/history was not enumerated. See 08-database-documentation.md §8.4.
- **apps/api/Dockerfile** — referenced by railway.json but its own contents (base image, build stages) were not read in this pass.
- **apps/api/prisma/seed.ts** — referenced by package.json’s seed script; contents not read.
- **apps/api/vitest.config.ts / vitest.integration.config.ts** — existence and role confirmed via the npm scripts that invoke them; internal configuration (coverage settings, test file globs, timeouts) not read.

24.3 Explicit exclusions by design of this pack

- **No destructive or exploit testing was performed** anywhere in preparing this pack, per the brief’s own instruction. All security-relevant statements are therefore necessarily “what the code appears to do,” not “what was proven to hold under attack.”
- **No .env.example or environment-secret file was located and none was created** — this pack does not, and should not, contain real secret values (see 16-configuration-and-environment.md).
- **This pack does not redesign, recommend, or propose changes** — every observation of a gap or risk in 20-known-issues-risks-technical-debt.md is reported as

an as-built characteristic, not accompanied by a prescribed fix, per the “document what exists, not what should exist” instruction.

24.4 Recommended follow-up (if a fuller pass is wanted later)

For completeness, without implying these are urgent or that this pack is deficient for lacking them within its current scope: a follow-up pass could (a) read every remaining controller and modal/form component individually to upgrade the INFERRED UI/UX details in doc 06 to CONFIRMED, (b) enumerate the prisma/migrations directory for doc 08 §8.4, (c) read the assertion bodies of all 16 integration test files to produce a scenario-level (not file-level) coverage matrix for doc 12, and (d) run npm audit/an equivalent scanner to close the dependency-vulnerability gap noted in doc 17 §17.6.